Image transformations with PyTorch

Operator manual | lorem-warped | Version 1.4

This manual uses selected writing principles from ASD-STE100 Issue 9. It is not certified ASD-STE100 documentation. Formal conformity needs a controlled-dictionary and rule review.

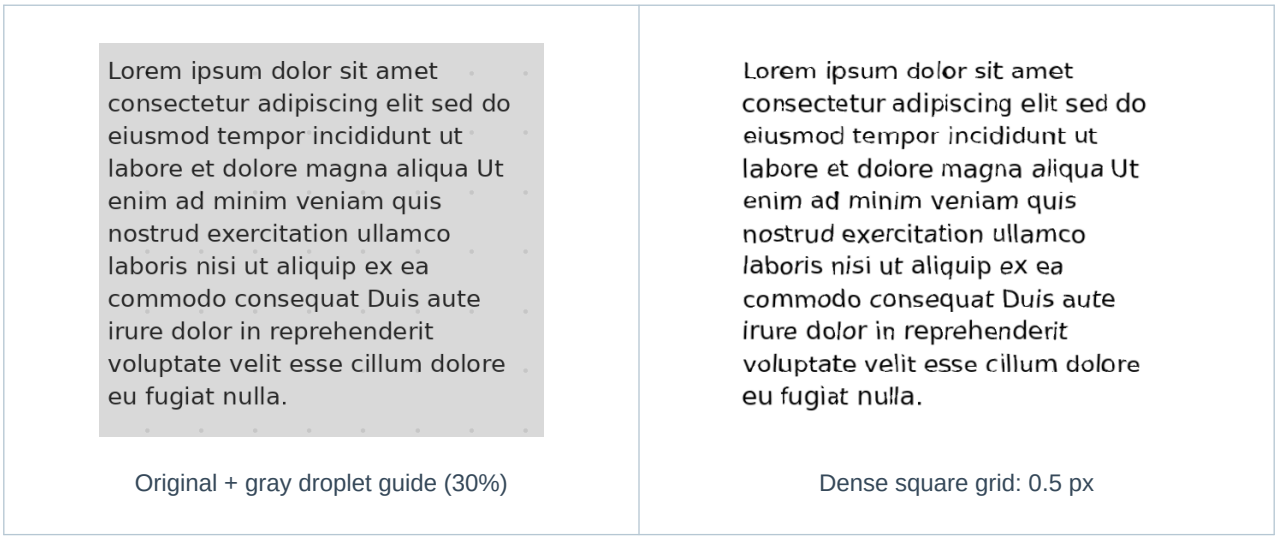
1. Package introduction

lorem-warped applies independent PyTorch image transformations to a single RGB image. It includes droplet layouts, page waves, a six-control sine lens, a gentle pinch, a refractive-index slab, and a Python-only custom callback. Input and output are CHW tensors; the CLI writes PNG files.

This document includes the input-generation recipe, controls, code, and images for each transform. The shown input is a 1200-by-1600 RGB page with a 28 px body font; the following setup recreates it exactly. All study settings are printed beside their images.

1.1 Distortion overview

A distortion moves source pixels to new positions. The left image is the unwarped original with its actual droplet regions shaded gray at 30% opacity. The right image uses a dense square-grid droplet warp. The guide is display-only: it is not drawn on the transform input or output.

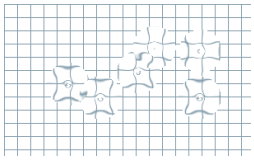
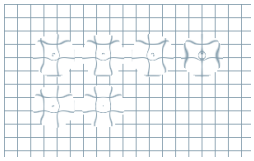
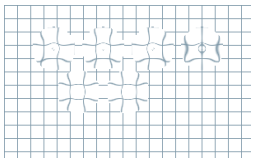
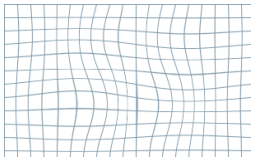
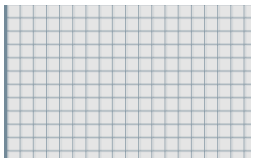
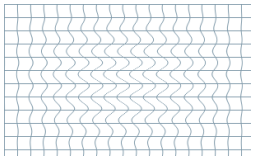
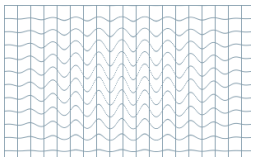
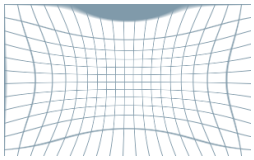
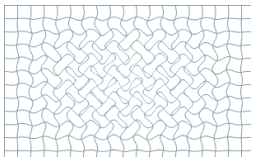


For this input, request 392 square-grid centres, a radius of 70 pixels, and an intensity of 0.5 pixels. This low intensity keeps the dense example legible. More centres make the grid denser. Radius sets each droplet's size. No droplet example in this manual exceeds 5 pixels of intensity.

Choose a transform: droplets give local circles; a sine lens and other page warps move larger page regions; refraction makes two translated views. Choose random, square, or triangle (hexagonal) centres for droplets. Set centre count, radius, intensity, seed, and profile. The other modes do not use centre distribution.

2. Distortion types

Read this tree from left to right. The middle column applies each actual transform to the same dense, text-free square grid. The right column applies it to the same short paragraph. Grid lines move with the transform; they are not droplet centres. Droplet layouts use 3 px intensity, and a uniform slab shifts lines without bending them.

Type	Distorted square grid	Distorted text
`|-- droplets` `|-- random`		Lorem ipsum dolor sit amet consectetur adipiscing elit
`|  |-- square grid`		Lorem ipsum dolor sit amet consectetur adipiscing elit
`|  \`-- triangular grid`		Lorem ipsum dolor sit amet consectetur adipiscing elit
`|-- Sine lens`		Lorem ipsum dolor sit amet consectetur adipiscing elit
`|-- Refractive-index slab`		Lorem ipsum dolor sit amet consectetur adipiscing elit
`|-- Horizontal wave`		Lorem ipsum dolor sit amet consectetur adipiscing elit
`|-- Vertical wave`		Lorem ipsum dolor sit amet consectetur adipiscing elit
`|-- Gentle pinch`		Lorem ipsum dolor sit amet consectetur adipiscing elit
`|-- Custom wave`		Lorem ipsum dolor sit amet consectetur adipiscing elit

The slab view is a uniform translation of two polarization modes. Its preview does not imply a spatial lens. A custom callback has no CLI command.

3. Install and prepare

Install Python 3.10 or later. Open a shell in the lorem-warped source checkout root, create an environment, and install the project dependencies. Keep this directory as the working directory: the recipes import the src package from the checkout. No separate sample image or study file is required.

a. CLI

```
python -m venv .venv
. .venv/bin/activate
python -m pip install -e '.[dev]'
python -m src --help
```

On Windows PowerShell, activate .venv\Scripts\Activate.ps1. Use the PyTorch installation selector if you need a specific GPU build.

Create the image shown in the full-page studies with the package's built-in page renderer. Its 28 px body text is the basis of the 14, 28, 42, 56, and 70 px periods. Create it before either the CLI or script recipes. You may substitute your own RGB image; keep masks the same size as that image.

b. CLI: create the input

```
python -c "from src.demo import render_lorem_page; render_lorem_page().save('original.png')"
```

4. Script setup

In script examples, read the image once. The following setup also creates the input if absent. Each transform starts from source, not from a previous output. PIL images become float CHW tensors from 0 to 1.

a. in script

```
from PIL import Image
from src.demo import render_lorem_page
from src import (CentreDistribution, DistortionConfig,
                 WarpTransform, image_to_tensor)
render_lorem_page().save('original.png')
source = image_to_tensor(Image.open('original.png'))
def save(path, result):
    pixels = result.clamp(0, 1).mul(255).round().byte()
    pixels = pixels.permute(1, 2, 0).cpu().numpy()
    Image.fromarray(pixels).save(path)
```

The custom-wave example imports partial and the supplied custom_wave callback. The CLI runs on CPU. Move the tensor to a supported GPU in your script if necessary.

Geometric fields use pixel displacement. The sampler reads source pixels at output position minus intensity times field. It uses bilinear sampling, border padding, and aligned corners. Strong fields can fold or repeat text. If the package cannot be imported, activate the environment and install it from the project root. For a different physical page size, change both page dimensions in the refractive recipe.

4.1 Create the four-line comparison source

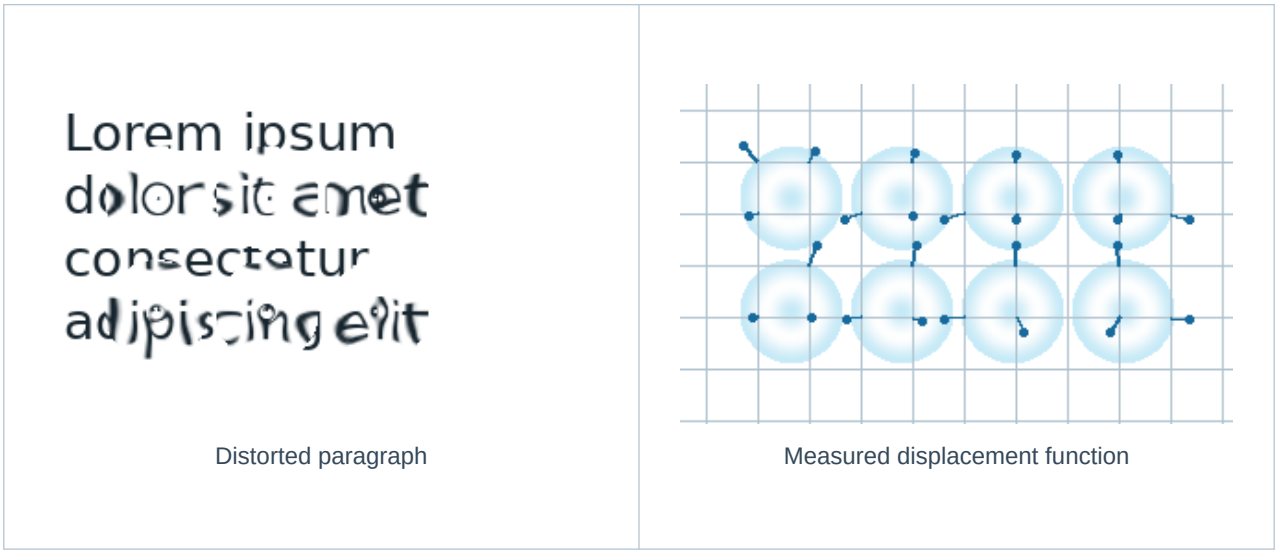
The compact 4-by-4 studies use this separate 300-by-184 source. Create it once. The straight reference grid is part of the source tensor, so text and lines are warped together. The full-page five-by-five studies instead use original.png.

a. in script

```
from PIL import ImageDraw, ImageFont
from src import overlay_warped_grid
mini = Image.new('RGB', (300, 184), 'white')
pen = ImageDraw.Draw(mini)
font = ImageFont.truetype('DejaVuSans.ttf', 28)
for row, line in enumerate(('Lorem ipsum', 'dolor sit amet',
                            'consectetur', 'adipiscing elit')):
    pen.text((10, 10 + row*34), line, font=font,
             fill='#17252e')
mini.save('mini-source.png')
mini_grid = overlay_warped_grid(image_to_tensor(mini),
                                spacing=28, opacity=.24)
save('mini-grid.png', mini_grid)
```

5. How droplet transformation works

Each centre has a radius. The cosine profile gives a radial displacement that fades to zero at the circle edge. Overlapping fields add before the image is sampled once. Pixel intensity multiplies the field. The right-hand plot shows the measured displacement vectors and shaded influence on a square reference grid.



Distorted paragraph

Measured displacement function

The left panel uses square; 8 centres; radius 28 px; intensity 2.5 px. The plot shows its actual pixel displacement on a square sampling grid. Arrow lengths are amplified for display. The plot is not an extra transformation.

5.1 Droplet transformation: strength examples

A droplet warp moves pixels around circular centres. Choose random, square, or triangle (hexagonal) centres. Radius sets each circle size. Intensity sets the movement in pixels. The seed fixes random centres. The cosine profile permits overlaps.

The first panel is the unwarped original with display-only gray droplet guides at 30% opacity. The other panels warp the unmarked source. Only pixel intensity changes; no example exceeds 5 pixels.



5.2 Run droplet transformation

a. CLI

```
python -m src --input original.png \
  --output droplet.png --centres 3 --distribution random \
  --radius 80 --intensity 5 --seed 7 --profile cosine
```

b. in script

```
transform = WarpTransform(mode='droplet',
    config=DistortionConfig(centre_count=3, radius=80,
                            intensity=5, seed=7))
save('droplet.png', transform(source))
```

The opening overview uses a dense square arrangement at 0.5 px intensity. Its gray guide is display-only; the distorted image is generated by these settings.

c. CLI: overview

```
python -m src --input original.png \
  --centres 392 --distribution square \
  --radius 70 --intensity 0.5 --seed 7 \
  --profile cosine --output dense-overview.png
```

d. in script: overview

```
config = DistortionConfig(centre_count=392,
    distribution=CentreDistribution.SQUARE,
    radius=70, intensity=0.5, seed=7,
    profile='cosine')
save('dense-overview.png',
     WarpTransform(mode='droplet', config=config)(source))
```

The compact study uses the mini-source.png text from section 4.1, eight square-distributed centres, and the listed radius/intensity pairs. Unlike page-warps, droplet samples do not contain the straight reference grid. Gray circles on zero-warp controls are display-only annotations.

e. CLI: four-by-four study

```
for radius in 30 45 60 75; do
  for i in 0 1 3 5; do
    python -m src --input mini-source.png \
      --centres 8 --distribution square --seed 7 \
      --profile cosine --radius $radius --intensity $i \
      --output droplet_${radius}_${i}.png
  done
done
```

f. in script: four-by-four study

```
small = image_to_tensor(Image.open('mini-source.png'))
for radius in (30, 45, 60, 75):
    for i in (0, 1, 3, 5):
        config = DistortionConfig(centre_count=8,
            distribution=CentreDistribution.SQUARE,
            radius=radius, intensity=i, seed=7,
            profile='cosine')
        save(f'droplet_{radius}_{i}.png',
             WarpTransform(mode='droplet',
                           config=config)(small))
```

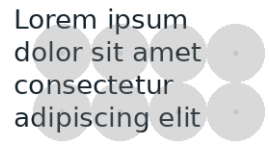
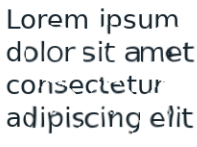
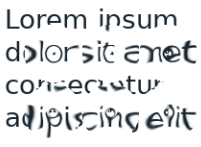
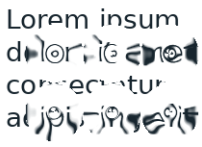
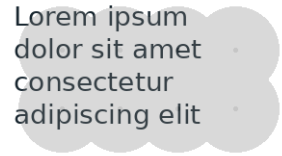
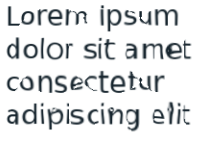
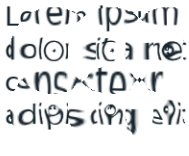
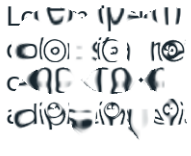
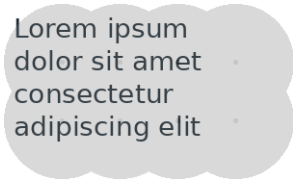
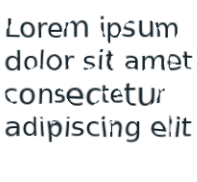
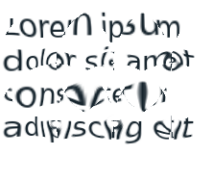
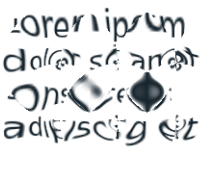
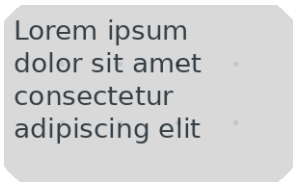
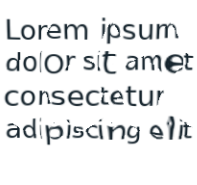
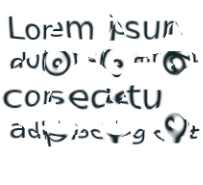
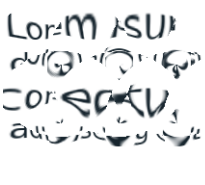
The command writes a PNG. On PowerShell, enter the CLI command on one line. Do not use Unix backslash line continuation there.

5.3 Two-variable legibility study

Rows change radius (px); columns change intensity (px). The 4-by-4 table uses the same four-line source paragraph, unchanged in font, size, and style. Each label lists the row value, then the column value. Compare the letters, spaces, and repeated strokes.

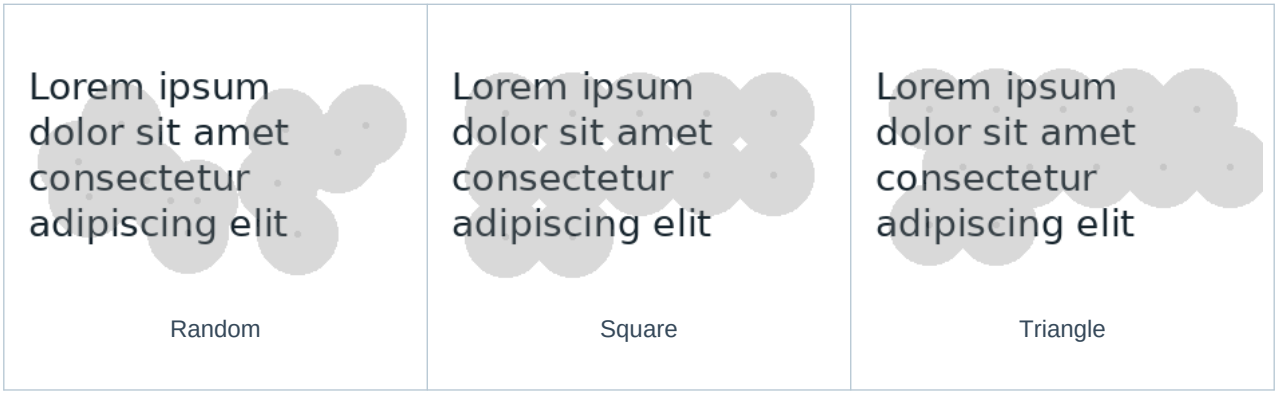
Zero-warp controls show gray droplet regions at 30% opacity. Other cells use actual droplet warps at no more than 5 pixels of intensity.

Keep all other transform settings fixed. The first column includes a zero-intensity control where applicable.

 30 | 0	 30 | 1	 30 | 3	 30 | 5
 45 | 0	 45 | 1	 45 | 3	 45 | 5
 60 | 0	 60 | 1	 60 | 3	 60 | 5
 75 | 0	 75 | 1	 75 | 3	 75 | 5

5.4 Shaded droplet regions

Each gray circle has 30% opacity and shows one droplet's radius. Each gray dot marks a centre. The same original text lies under every shaded map. These are centre-placement guides, not warped output images.



Random uses the seed to place centres. Square uses aligned rows and columns. Triangle offsets alternate rows to make a hexagonal neighbour pattern. The shaded regions can overlap with a cosine profile; parabolic profiles need nonoverlapping centres.

5.5 Droplet arrangements

The distribution controls centre positions. Random selects seeded positions. Square places centres in aligned rows and columns. Triangle offsets alternate rows and makes a hexagonal neighbour pattern. In the CLI, select the hexagonal pattern with --distribution triangle, not hex.

Compare the same text fragment. Each row pairs an unwarped original with its actual droplet regions shaded gray at 30% opacity (left) and the warped output (right). Request 71 centres for each layout. Keep radius 50 pixels, intensity 3 pixels, seed 7, and the cosine profile fixed. Only the distribution changes. The overlays are display-only. A shifted triangle row can have fewer centres near an edge.



5.6 Select a droplet distribution

Use one of these three names. Keep all other controls fixed to compare the centre layouts. The CLI commands below use the count and radius shown on the preceding page.

random. CLI

```
python -m src --input original.png \
  --output droplets_random.png --centres 71 \
  --distribution random --radius 50 \
  --intensity 3 --seed 7 --profile cosine
```

square. CLI

```
python -m src --input original.png \
  --output droplets_square.png --centres 71 \
  --distribution square --radius 50 \
  --intensity 3 --seed 7 --profile cosine
```

triangle. CLI

```
python -m src --input original.png \
  --output droplets_triangle.png --centres 71 \
  --distribution triangle --radius 50 \
  --intensity 3 --seed 7 --profile cosine
```

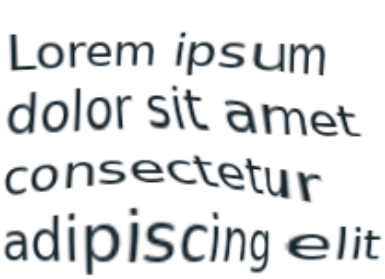
d. in script

```
for name in ('random', 'square', 'triangle'):
    config = DistortionConfig(centre_count=71,
        distribution=CentreDistribution(name), radius=50,
        intensity=3, seed=7, profile='cosine')
    save('droplets_' + name + '.png',
         WarpTransform(mode='droplet', config=config)(source))
```

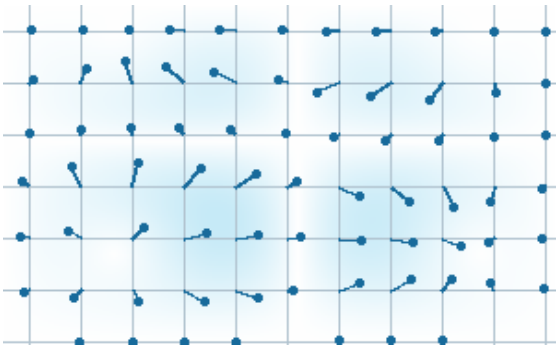
The script uses the imports and save function from section 4. A fixed centre count controls square and triangle density. A fixed seed repeats the random pattern. The cosine profile permits overlaps. A parabolic profile requires disjoint circles; --nonoverlapping-page makes disjoint page-wide layouts and overrides --centres.

6. How six-parameter sine lens works

The sine lens combines a two-axis sinusoidal envelope with local cell-centre directions. X and Y frequencies set the cell counts. Phase moves the lobes; axis gains set the X and Y movement. Pixel intensity multiplies the field.

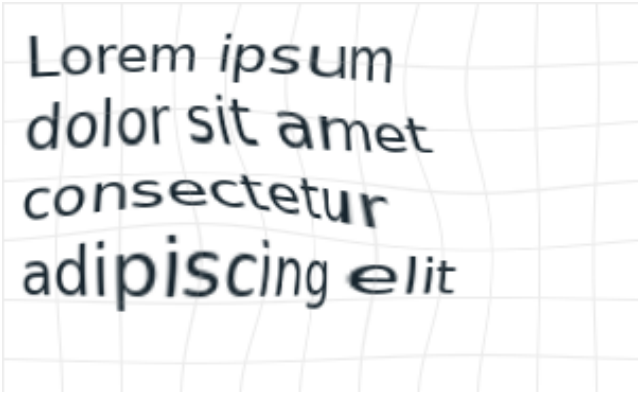


Distorted paragraph



Measured displacement function

The left panel uses X/Y frequency 1.5/1.5; intensity 80 px. The plot shows its actual pixel displacement on a square sampling grid. Arrow lengths are amplified for display. The plot is not an extra transformation.

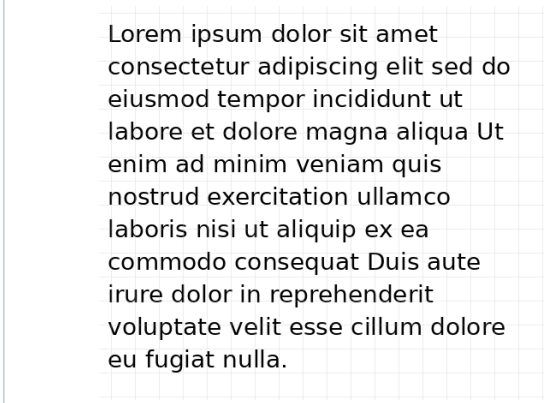


The small overlay above uses a dense square reference grid to show how this effect samples the image. It is not a square array of droplet centres. For the slab, the two mode translations remain constant across the page.

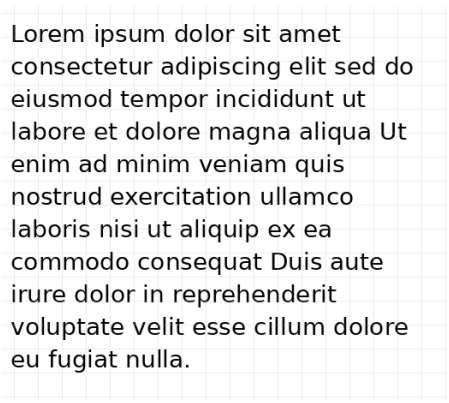
6.1 Six-parameter sine lens: strength examples

A sine lens moves the page in two axes. X and Y frequencies, phases, and gains give six shape parameters. The intensity multiplies the displacement in pixels. Keep the six parameters fixed in this comparison.

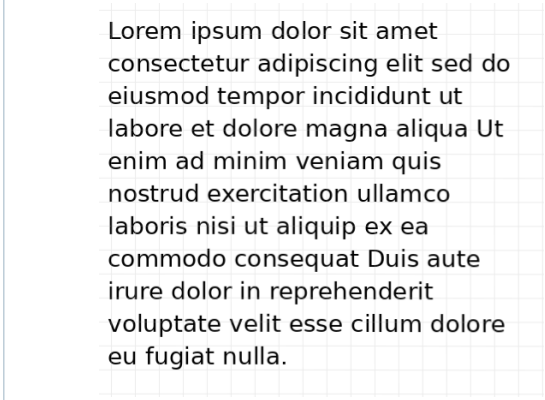
The first panel shows a straight, dense square grid over the original. Each output uses one sine_lens transform on both the text and this grid; the lines bend or translate with the text. The plain, ungridded output remains available to CLI and script users. Within these panels only px intensity changes. Strong settings may reduce legibility.



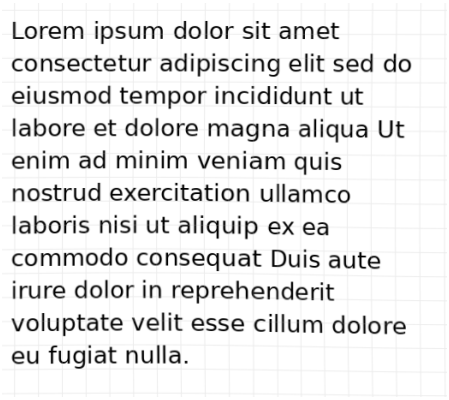
Original + straight square grid



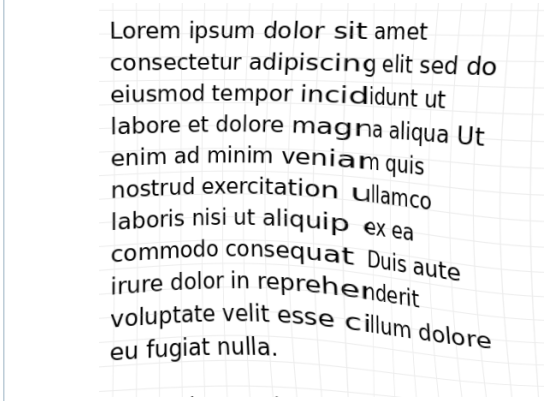
0.5 px



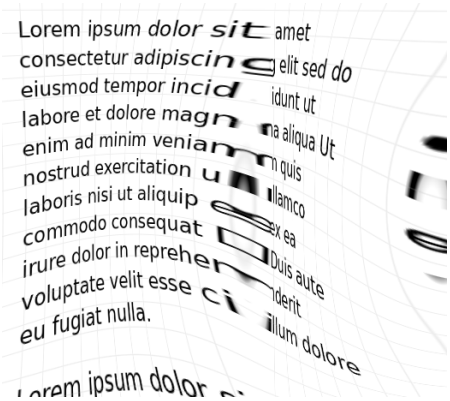
4 px



32 px



256 px



1024 px

6.2 Run six-parameter sine lens

Set the six lens parameters to X/Y frequencies 2.5/1.5, phases 0.1/0.2 cycles, and gains 1.2/0.8. Use 32 pixels for this example. A frequency must be positive.

a. CLI

```
python -m src --input original.png \
  --output sine_lens.png --page-warp sine_lens \
  --intensity 32 --lens-frequency-x 2.5 \
  --lens-frequency-y 1.5 --lens-phase-x 0.1 \
  --lens-phase-y 0.2 --lens-amplitude-x 1.2 \
  --lens-amplitude-y 0.8
```

b. in script

```
transform = WarpTransform(mode='sine_lens', intensity=32,
    lens_options=dict(frequency_x=2.5, frequency_y=1.5,
        phase_x=0.1, phase_y=0.2,
        amplitude_x=1.2, amplitude_y=0.8))
save('sine_lens.png', transform(source))
```

To reproduce the four-by-four frequency/intensity study, start from mini-grid.png. Hold Y frequency 1.5, X/Y phases 0.1/0.2 and gains 1.2/0.8 fixed.

c. CLI: four-by-four study

```
for f in 1 1.5 2 2.5; do
  for i in 0 15 40 80; do
    python -m src --input mini-grid.png \
      --page-warp sine_lens --lens-frequency-x $f \
      --lens-frequency-y 1.5 --lens-phase-x .1 \
      --lens-phase-y .2 --lens-amplitude-x 1.2 \
      --lens-amplitude-y .8 --intensity $i \
      --output lens_${f}_${i}.png
  done
done
```

d. in script: four-by-four study

```
small = image_to_tensor(Image.open('mini-grid.png'))
for f in (1, 1.5, 2, 2.5):
    for i in (0, 15, 40, 80):
        opts = dict(frequency_x=f, frequency_y=1.5,
            phase_x=.1, phase_y=.2,
            amplitude_x=1.2, amplitude_y=.8)
        save(f'lens_{f}_{i}.png',
             WarpTransform(mode='sine_lens',
                 intensity=i, lens_options=opts)(small))
```

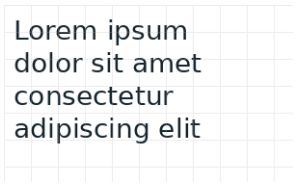
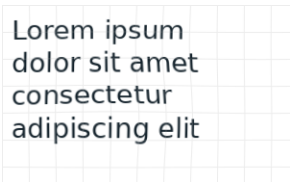
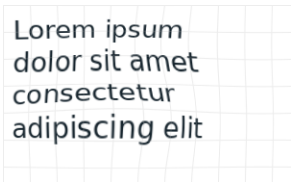
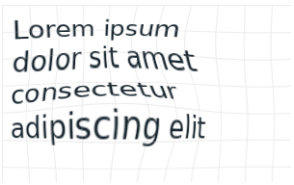
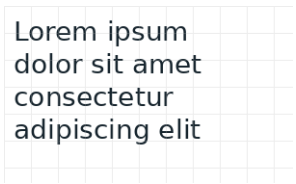
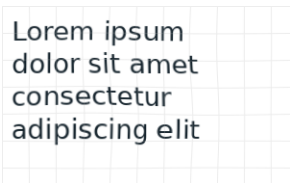
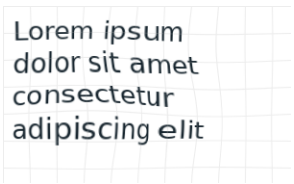
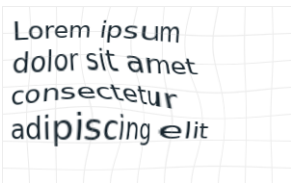
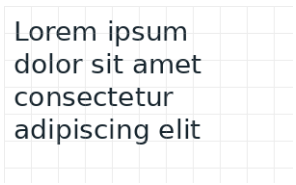
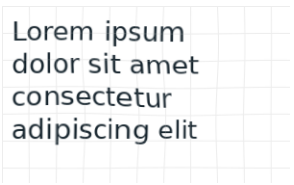
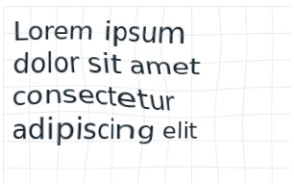
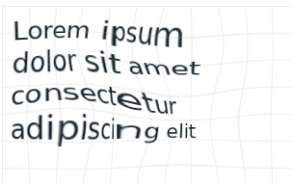
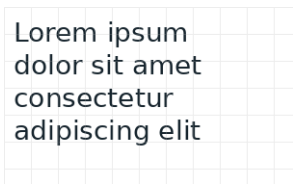
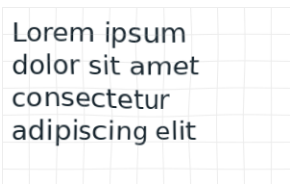
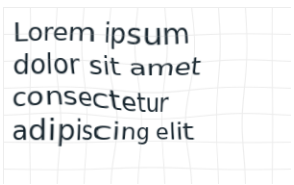
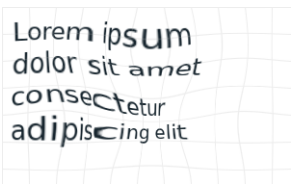
The command writes a PNG. On PowerShell, enter the CLI command on one line. Do not use Unix backslash line continuation there.

6.3 Two-variable legibility study

Rows change X frequency; columns change intensity (px). The 4-by-4 table uses the same four-line source paragraph, unchanged in font, size, and style. Each label lists the row value, then the column value. Compare the letters, spaces, and repeated strokes.

Every cell starts with the same dense square reference grid. The transform warps the grid with the text; zero-intensity cells keep straight lines. These lines do not represent droplet centres.

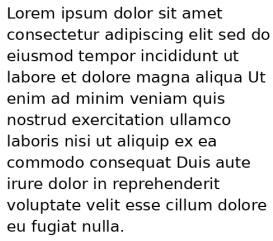
Keep all other transform settings fixed. The first column includes a zero-intensity control where applicable.

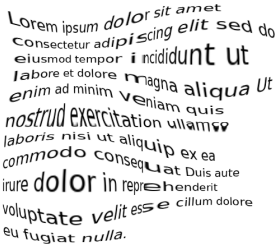
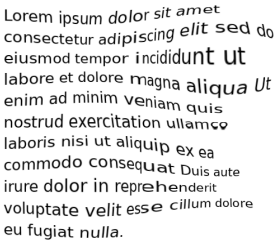
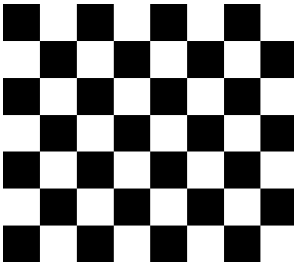
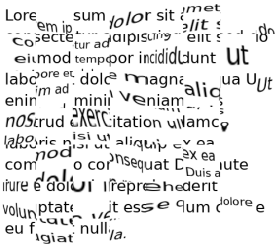
 1 | 0	 1 | 15	 1 | 40	 1 | 80
 1.5 | 0	 1.5 | 15	 1.5 | 40	 1.5 | 80
 2 | 0	 2 | 15	 2 | 40	 2 | 80
 2.5 | 0	 2.5 | 15	 2.5 | 40	 2.5 | 80

6.4 Mask the displacement

Use a grayscale mask with exactly the same width and height as the image. At each output pixel, black (0) cancels both displacement components; white (255) keeps the full field. Gray scales both components continuously: 128 gives about half strength. The sampler reads the original at the resulting coordinates. It does not blend an unwarped and a warped image, and it does not change image alpha.

The three results use the same text crop and sine-lens field at 128 px intensity. Row 1 compares the original with an unmasked warp. Row 2 shows a left-to-right gradient mask and its result. Row 3 shows a black-and-white checkerboard mask and its result; each square has side length one-seventh of the crop height in both axes. Only the mask changes. The following recipes create both masks from the input image; no separate mask file is required.

<table>
<tr>
<td> 

Original text</td>
<td> 
Without mask</td>
</tr>
<tr>
<td> 
Gradient mask</td>
<td> 
With gradient mask</td>
</tr>
<tr>
<td> 
Checkerboard mask</td>
<td> 
With checkerboard mask</td>
</tr>
</table>

Use the recipes in 6.5 to render this exact crop, both masks, and their warped results. A mask must match its input; it is never resized. The mask also works with droplets, page/custom warps, and refraction. Refraction retains optical transmission.

6.5 Reproduce the mask comparison

This script extracts the same 520-by-460 crop from the generated original.png. It creates both grayscale masks and produces all three transformed views shown in 6.4. Run it after the imports and save helper in section 4.

a. in script

```
from pathlib import Path
import numpy as np
crop = Image.open('original.png').convert('RGB')
crop = crop.crop((70, 104, 590, 564))
crop.save('mask-source.png')
w, h = crop.size
gradient = Image.new('L', (w, 1))
gradient.putdata([round(255*x/(w-1)) for x in range(w)])
gradient.resize((w, h)).save('gradient-mask.png')
y, x = np.indices((h, w))
checks = ((y*7//h + x*7//h) % 2 * 255).astype('uint8')
Image.fromarray(checks, 'L').save('checks-mask.png')
small = image_to_tensor(crop)
for name, mask in (('plain', None),
                   ('gradient', Path('gradient-mask.png')),
                   ('checks', Path('checks-mask.png'))):
    save('mask-' + name + '.png',
         WarpTransform(mode='sine_lens', intensity=128,
                       mask=mask)(small))
```

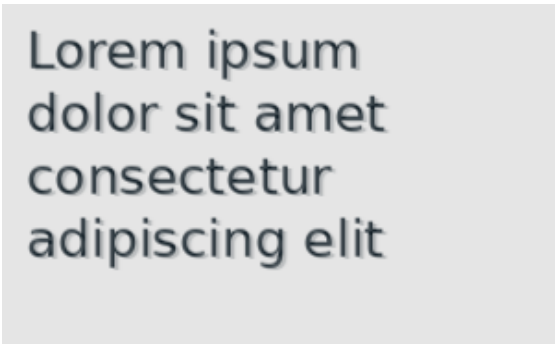
After the script creates mask-source.png and the two masks, these equivalent CLI commands generate the three views without any bundled example assets. Each command starts from the same source crop.

b. CLI

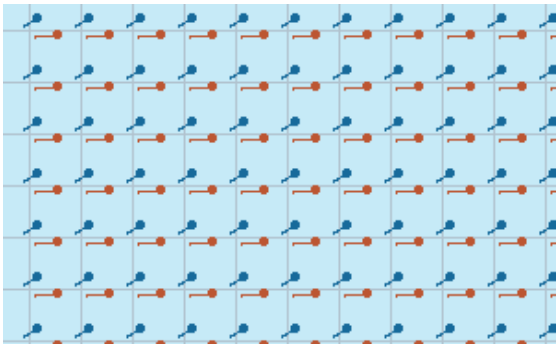
```
python -m src --input mask-source.png --page-warp sine_lens \
  --intensity 128 --output mask-plain.png
python -m src --input mask-source.png --page-warp sine_lens \
  --intensity 128 --mask gradient-mask.png \
  --output mask-gradient.png
python -m src --input mask-source.png --page-warp sine_lens \
  --intensity 128 --mask checks-mask.png \
  --output mask-checks.png
```

7. How refractive-index transformation works

The uniform slab yields two distinct constant translations: one for each polarization mode. Thickness changes their offsets. Polarization changes their weights. The right-hand plot shows both vectors, not a varying lens field. Displacement is converted from millimetres with the page dimensions.

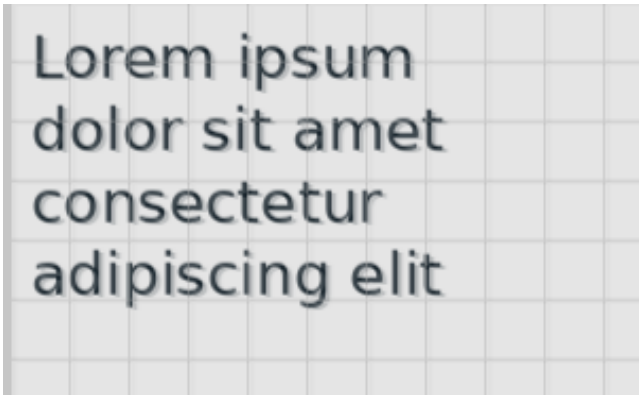


Distorted paragraph



Measured displacement function

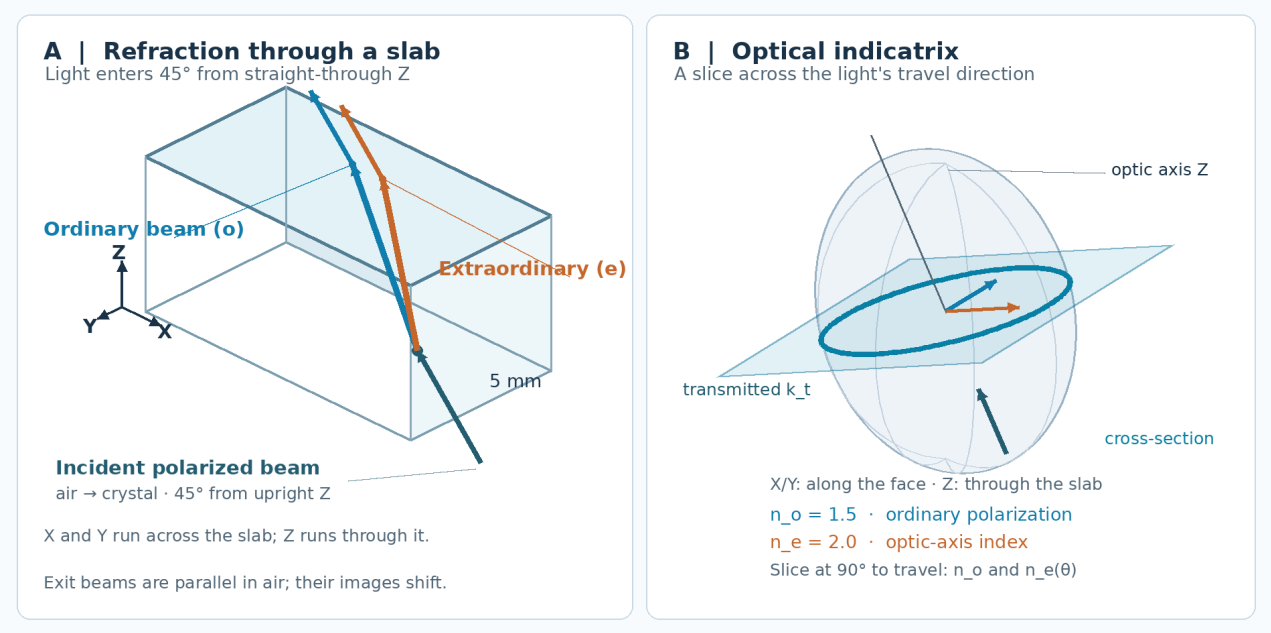
The left panel uses thickness 5 mm; polarization 30 degrees. The plot shows its actual pixel displacement on a square sampling grid. Arrow lengths are amplified for display. The plot is not an extra transformation.



The small overlay above uses a dense square reference grid to show how this effect samples the image. It is not a square array of droplet centres. For the slab, the two mode translations remain constant across the page.

7.0 Ray geometry and optical indicatrix

X and Y run along the slab's two flat edges; Z points through its thickness. The incoming beam tilts 45 degrees away from straight-through Z toward negative X, with no travel along Y: halfway between straight through and sideways. In this uniaxial example, the blue Ordinary beam and orange Extraordinary beam follow different paths, then leave parallel. Their paths come from the slab solver; the picture is not to scale.



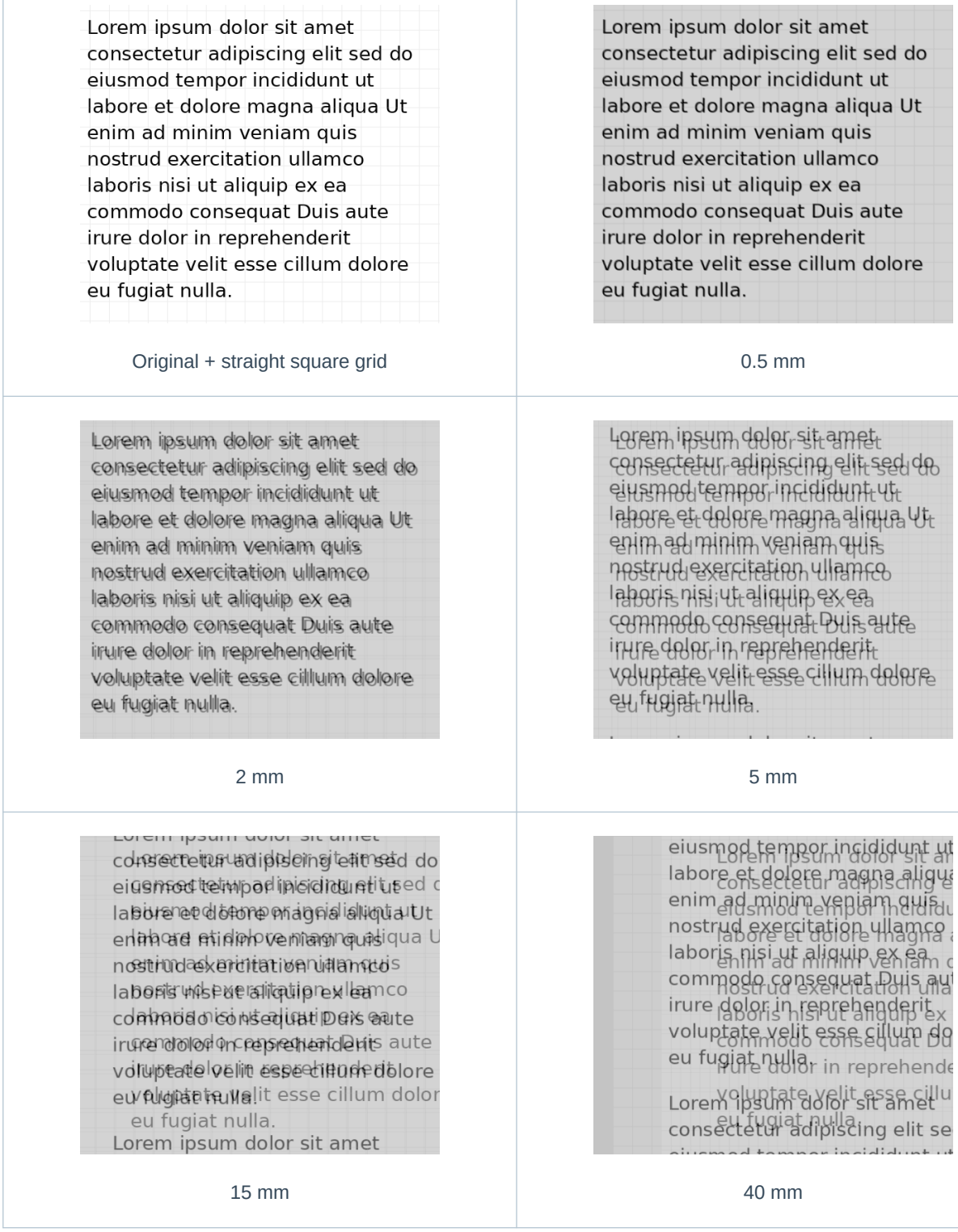
At right, imagine slicing the optical indicatrix (an egg-shaped index map) across the direction the light travels. This cyan cross-section is at 90 degrees to the transmitted wave normal; its colored arrows mark two ways the light can wiggle (polarization). Here n_x = n_y = 1.5, n_z = 2.0, thickness = 5 mm, and incident direction = (-1, 0, 1). The separate n_x-by-n_y studies can be biaxial, so they use X/Y eigenmode labels instead of ordinary/extraordinary.

The diagram shows the principal axes and the two polarization directions. The ray positions come from the slab solver; polarization weights at oblique incidence are approximate. The schematic and its parameters are completely described here.

7.1 Refractive-index transformation: strength examples

A uniform anisotropic slab makes two translated polarization views. Thickness sets the translation in millimetres. The refractive indices and physical page size stay fixed. Do not use pixel intensity for this physical model.

The first panel shows a straight, dense square grid over the original. Each output uses one refractive_index transform on both the text and this grid; the lines bend or translate with the text. The plain, ungridded output remains available to CLI and script users. Within these panels only mm thickness changes. Strong settings may reduce legibility.



7.2 Run refractive-index transformation

Thickness and page dimensions use millimetres. The angle uses degrees. The slab gives constant translations, not a spatially varying lens. The polarization weights are approximate at oblique incidence.

a. CLI

```
python -m src --input original.png \
  --output refraction.png --page-warp refractive_index \
  --refraction-thickness-mm 5 --refraction-n-x 1.3 \
  --refraction-n-y 2.5 --refraction-n-z 1.3 \
  --refraction-polarization-deg 45 \
  --refraction-direction -1 1 1 \
  --page-width-mm 210 --page-height-mm 297
```

b. in script

```
transform = WarpTransform(mode='refractive_index',
    refraction_options=dict(thickness_mm=5, n_x=1.3,
        n_y=2.5, n_z=1.3, polarization_deg=45,
        direction_xyz=(-1, 1, 1),
        page_width_mm=210, page_height_mm=297))
save('refraction.png', transform(source))
```

The three compact studies use thickness 2, 5 or 10 mm. In each one, rows and columns independently set n_x and n_y; n_z remains 1.3. No pixel intensity is used.

c. CLI: three four-by-four studies

```
for thickness in 2 5 10; do
  for nx in 1 1.2 1.5 2; do
    for ny in 1 1.2 1.5 2; do
      python -m src --input mini-grid.png \
        --page-warp refractive_index \
        --refraction-thickness-mm $thickness \
        --refraction-n-x $nx --refraction-n-y $ny \
        --refraction-n-z 1.3 \
        --refraction-polarization-deg 45 \
        --refraction-direction -1 1 1 \
        --page-width-mm 210 --page-height-mm 297 \
        --output slab_${thickness}_${nx}_${ny}.png
    done
  done
done
```

d. in script: three studies

```
small = image_to_tensor(Image.open('mini-grid.png'))
for thickness in (2, 5, 10):
    for nx in (1, 1.2, 1.5, 2):
        for ny in (1, 1.2, 1.5, 2):
            opts = dict(thickness_mm=thickness,
                n_x=nx, n_y=ny, n_z=1.3,
                polarization_deg=45,
                direction_xyz=(-1, 1, 1),
                page_width_mm=210, page_height_mm=297)
            save(f'slab_{thickness}_{nx}_{ny}.png',
                WarpTransform(mode='refractive_index',
                    refraction_options=opts)(small))
```

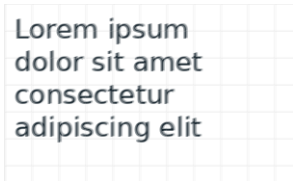
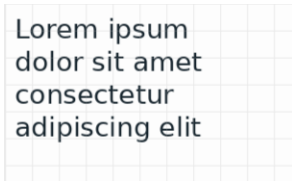
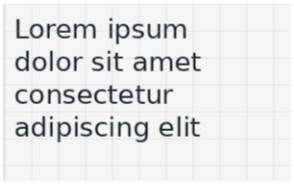
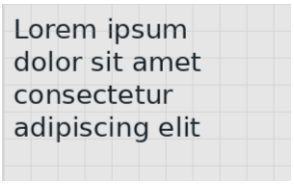
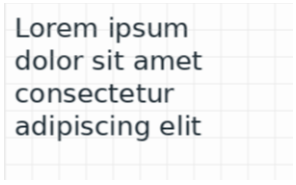
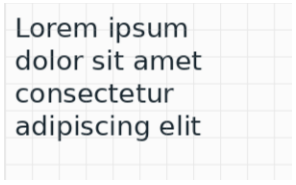
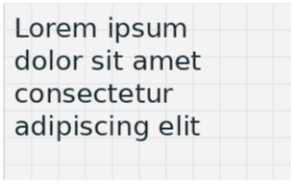
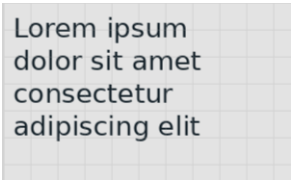
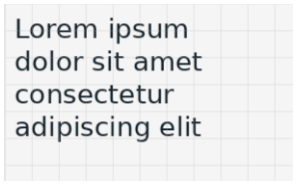
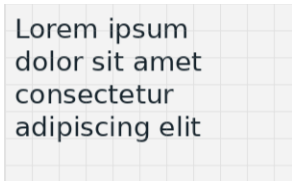
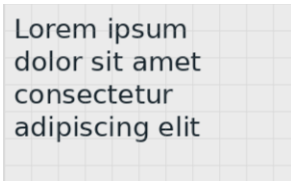
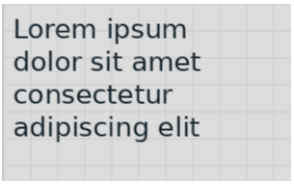
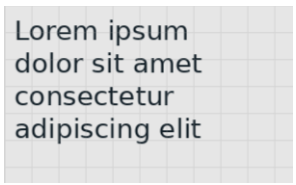
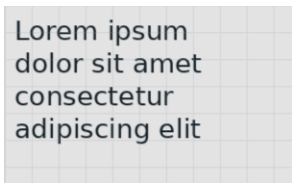
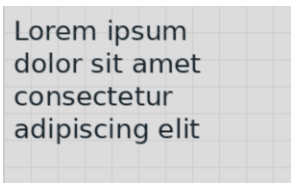
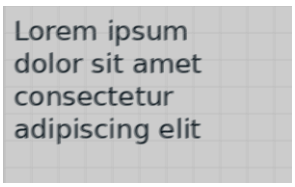
The command writes a PNG. On PowerShell, enter the CLI command on one line. Do not use Unix backslash line continuation there.

7.3 Two-variable legibility study: 2 mm

Rows change n_x; columns change n_y. The 4-by-4 table uses the same four-line source paragraph, unchanged in font, size, and style. Each label lists the row value, then the column value. Compare the letters, spaces, and repeated strokes.

Every cell starts with the same dense square reference grid. The uniform slab translates the grid with the text; grid lines stay straight, though their contrast can change. These lines do not represent droplet centres.

Keep slab thickness at 2 mm and polarization at 45 degrees. Change n_x down the rows and n_y across the columns: 1, 1.2, 1.5, 2.0. The indices are dimensionless; n_z remains 1.3. A uniform slab produces translated views, not a curved lens. No artistic pixel intensity is used.

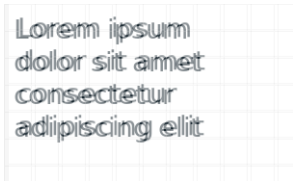
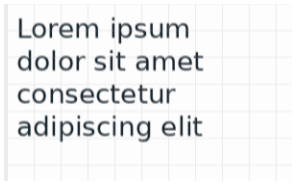
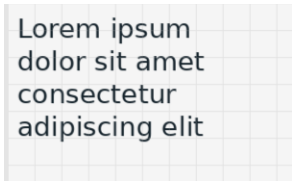
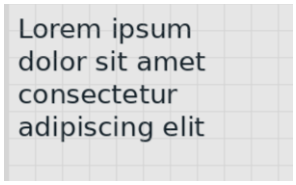
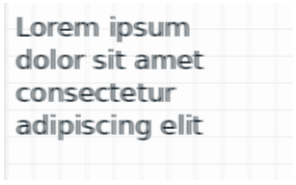
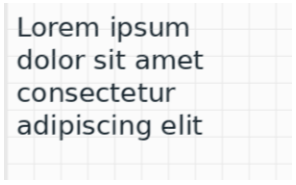
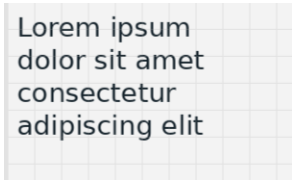
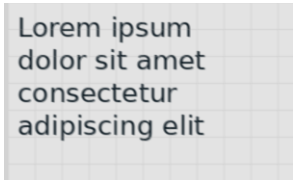
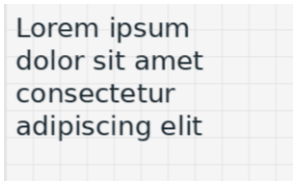
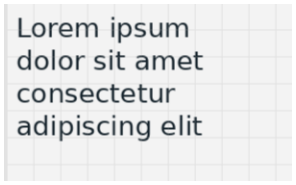
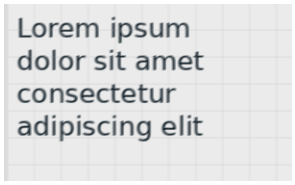
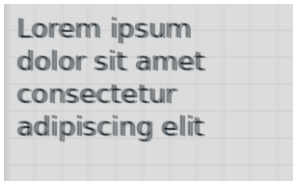
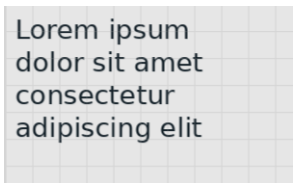
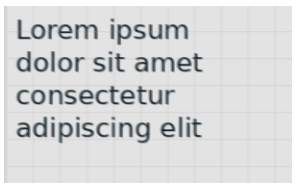
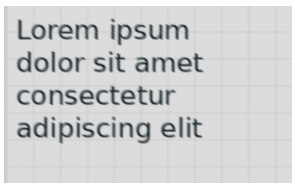
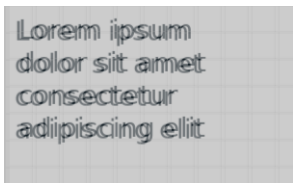
 1 | 1	 1 | 1.2	 1 | 1.5	 1 | 2
 1.2 | 1	 1.2 | 1.2	 1.2 | 1.5	 1.2 | 2
 1.5 | 1	 1.5 | 1.2	 1.5 | 1.5	 1.5 | 2
 2 | 1	 2 | 1.2	 2 | 1.5	 2 | 2

7.3 Two-variable legibility study: 5 mm

Rows change n_x; columns change n_y. The 4-by-4 table uses the same four-line source paragraph, unchanged in font, size, and style. Each label lists the row value, then the column value. Compare the letters, spaces, and repeated strokes.

Every cell starts with the same dense square reference grid. The uniform slab translates the grid with the text; grid lines stay straight, though their contrast can change. These lines do not represent droplet centres.

Keep slab thickness at 5 mm and polarization at 45 degrees. Change n_x down the rows and n_y across the columns: 1, 1.2, 1.5, 2.0. The indices are dimensionless; n_z remains 1.3. A uniform slab produces translated views, not a curved lens. No artistic pixel intensity is used.

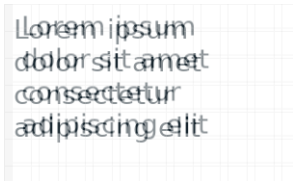
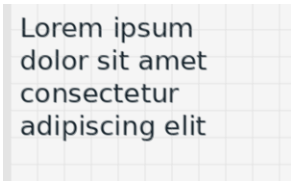
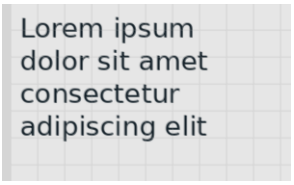
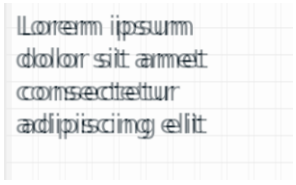
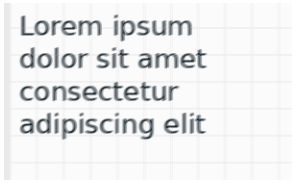
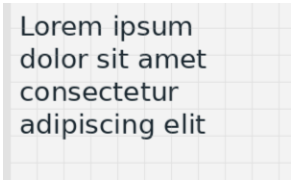
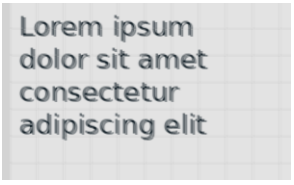
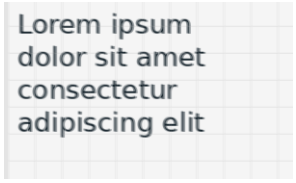
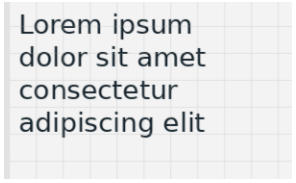
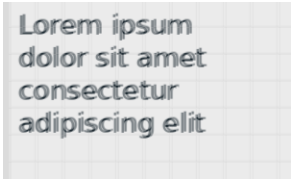
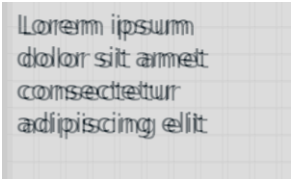
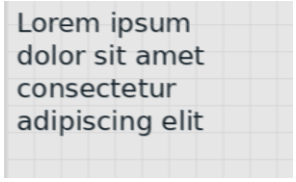
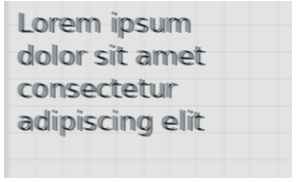
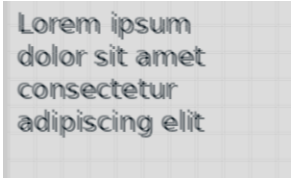
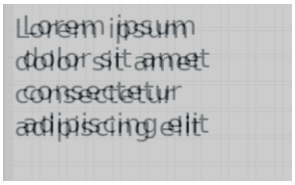
 1 | 1	 1 | 1.2	 1 | 1.5	 1 | 2
 1.2 | 1	 1.2 | 1.2	 1.2 | 1.5	 1.2 | 2
 1.5 | 1	 1.5 | 1.2	 1.5 | 1.5	 1.5 | 2
 2 | 1	 2 | 1.2	 2 | 1.5	 2 | 2

7.3 Two-variable legibility study: 10 mm

Rows change n_x; columns change n_y. The 4-by-4 table uses the same four-line source paragraph, unchanged in font, size, and style. Each label lists the row value, then the column value. Compare the letters, spaces, and repeated strokes.

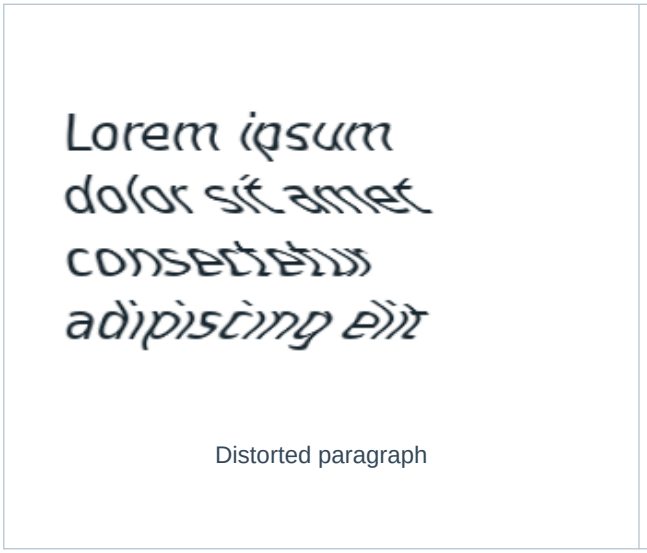
Every cell starts with the same dense square reference grid. The uniform slab translates the grid with the text; grid lines stay straight, though their contrast can change. These lines do not represent droplet centres.

Keep slab thickness at 10 mm and polarization at 45 degrees. Change n_x down the rows and n_y across the columns: 1, 1.2, 1.5, 2.0. The indices are dimensionless; n_z remains 1.3. A uniform slab produces translated views, not a curved lens. No artistic pixel intensity is used.

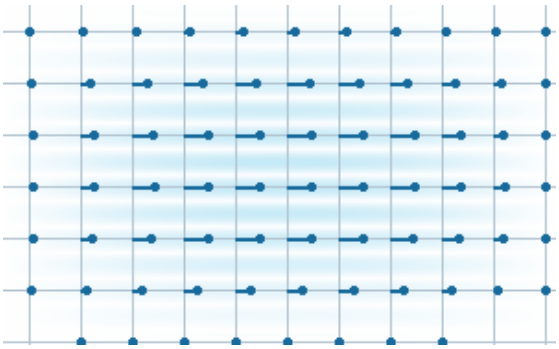
 1 | 1	 1 | 1.2	 1 | 1.5	 1 | 2
 1.2 | 1	 1.2 | 1.2	 1.2 | 1.5	 1.2 | 2
 1.5 | 1	 1.5 | 1.2	 1.5 | 1.5	 1.5 | 2
 2 | 1	 2 | 1.2	 2 | 1.5	 2 | 2

8. How horizontal wave works

The horizontal wave repeats along Y with a wavelength measured in source-image pixels. This gallery uses 28 px. The smooth edge envelope pins page edges. Pixel intensity multiplies the horizontal displacement.

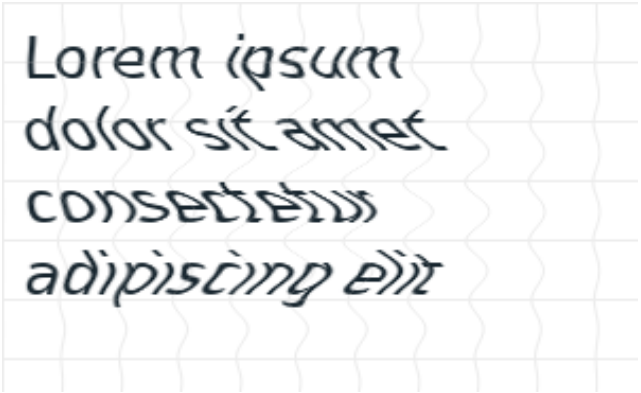


Distorted paragraph



Measured displacement function

The left panel uses Y wavelength 28 px; intensity 8 px. The plot shows its actual pixel displacement on a square sampling grid. Arrow lengths are amplified for display. The plot is not an extra transformation.



The small overlay above uses a dense square reference grid to show how this effect samples the image. It is not a square array of droplet centres. For the slab, the two mode translations remain constant across the page.

8.1 Horizontal wave: strength examples

A horizontal wave moves text left and right. Wavelength is the period along Y in pixels. These examples use 28 px. The page edges stay in place. Intensity sets movement in pixels.

The first panel shows a straight, dense square grid over the original. Each output uses one horizontal_wave transform on both the text and this grid; the lines bend or translate with the text. The plain, ungridded output remains available to CLI and script users. Within these panels only px intensity changes. Strong settings may reduce legibility.



8.2 Run horizontal wave

a. CLI

```
python -m src --input original.png \
  --output horizontal_wave.png --page-warp horizontal_wave --wavelength 28 \
  --intensity 20
```

b. in script

```
transform = WarpTransform(mode='horizontal_wave', wavelength=28, intensity=20)
save('horizontal_wave.png', transform(source))
```

To reproduce the compact 4-by-4 study, use the mini-grid.png source from section 4.1. Rows use 14, 28, 42, 56 px wavelengths; columns use 0, 12, 28, 50 px intensity. These commands apply the built-in transform to the text and straight grid together.

c. CLI: four-by-four study

```
for w in 14 28 42 56; do
  for i in 0 12 28 50; do
    python -m src --input mini-grid.png \
      --page-warp horizontal_wave --wavelength $w \
      --intensity $i --output study_horizontal_wave_${w}_${i}.png
  done
done
```

d. in script: four-by-four study

```
small = image_to_tensor(Image.open('mini-grid.png'))
for w in (14, 28, 42, 56):
    for i in (0, 12, 28, 50):
        transform = WarpTransform(mode='horizontal_wave',
            wavelength=w, intensity=i)
        save(f'study_horizontal_wave_{w}_{i}.png',
             transform(small))
```

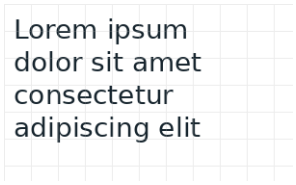
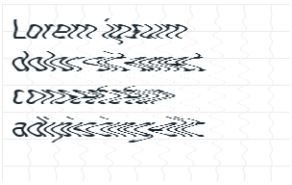
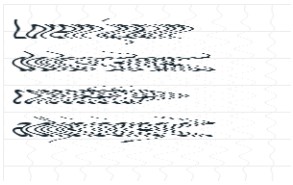
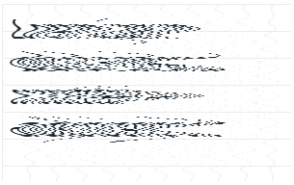
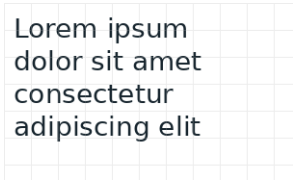
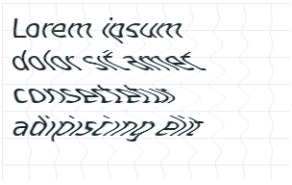
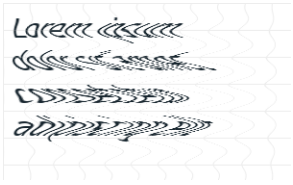
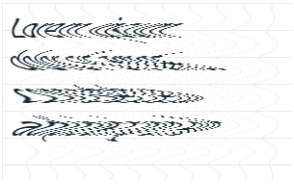
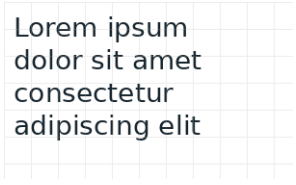
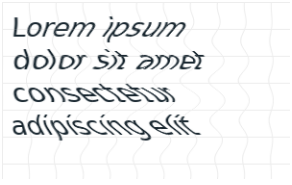
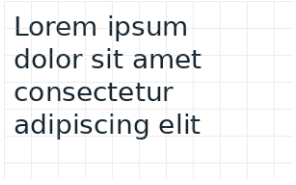
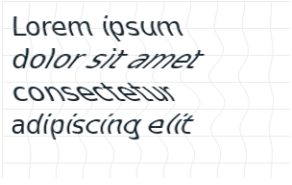
The command writes a PNG. On PowerShell, enter the CLI command on one line. Do not use Unix backslash line continuation there.

8.3 Two-variable legibility study

Rows change Y wavelength (px); columns change intensity (px). The 4-by-4 table uses the same four-line source paragraph, unchanged in font, size, and style. Each label lists the row value, then the column value. Compare the letters, spaces, and repeated strokes.

Every cell starts with the same dense square reference grid. The transform warps the grid with the text; zero-intensity cells keep straight lines. These lines do not represent droplet centres.

Each row uses a pixel wavelength calculated from the 28 px source font: 50%, 100%, 150%, and 200% give 14, 28, 42, and 56 px. The built-in wave renders both text and grid with that wavelength; columns vary pixel intensity. The following five-by-five study extends the sweep to 250% and 50 px intensity.

 14 px | 0 px	 14 px | 12 px	 14 px | 28 px	 14 px | 50 px
 28 px | 0 px	 28 px | 12 px	 28 px | 28 px	 28 px | 50 px
 42 px | 0 px	 42 px | 12 px	 42 px | 28 px	 42 px | 50 px
 56 px | 0 px	 56 px | 12 px	 56 px | 28 px	 56 px | 50 px

8.4 Two-parameter wavelength study: Horizontal wave

The unchanged input has a 28 px body font. Its 50%, 100%, 150%, 200%, and 250% periods are 14, 28, 42, 56, 70 px. Columns vary wavelength; rows vary intensity: 10, 20, 30, 40, 50 px. Each panel crops the same region of a full-page warp.

The built-in wave uses the stated pixel wavelength and keeps the page-edge envelope fixed.

a. CLI (POSIX shell)

```
for w in 14 28 42 56 70; do
  for i in 10 20 30 40 50; do
    python -m src --input original.png \
      --output horizontal_wave_${w}_${i}.png \
      --page-warp horizontal_wave --wavelength $w --intensity $i
  done
done
```

b. in script

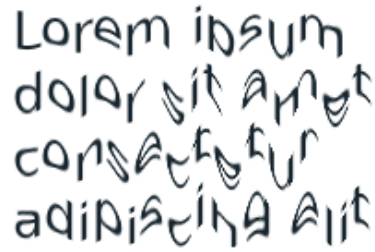
```
for w in (14, 28, 42, 56, 70):
    for i in (10, 20, 30, 40, 50):
        save(f'horizontal_wave_{w}_{i}.png',
             WarpTransform(mode='horizontal_wave',
                 wavelength=w, intensity=i)(source))
```



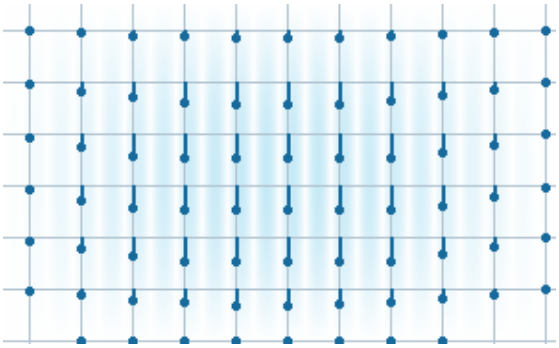
All row and column values appear on this page. Strong settings can fold or repeat letters.

9. How vertical wave works

The vertical wave repeats along X with a wavelength measured in source-image pixels. This gallery uses 28 px. The smooth edge envelope pins page edges. Pixel intensity multiplies the vertical displacement.

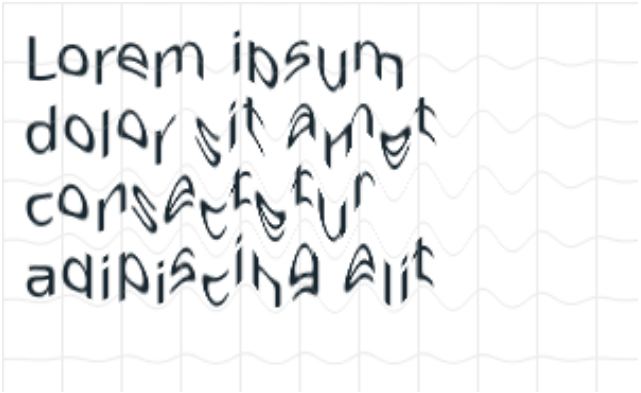


Distorted paragraph



Measured displacement function

The left panel uses X wavelength 28 px; intensity 10 px. The plot shows its actual pixel displacement on a square sampling grid. Arrow lengths are amplified for display. The plot is not an extra transformation.

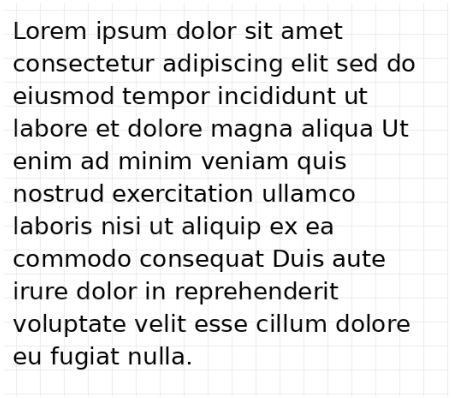


The small overlay above uses a dense square reference grid to show how this effect samples the image. It is not a square array of droplet centres. For the slab, the two mode translations remain constant across the page.

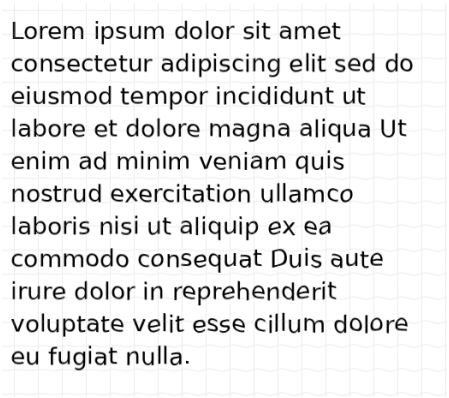
9.1 Vertical wave: strength examples

A vertical wave moves baselines up and down. Wavelength is the period along X in pixels. These examples use 28 px. The page edges stay in place. Intensity sets movement in pixels.

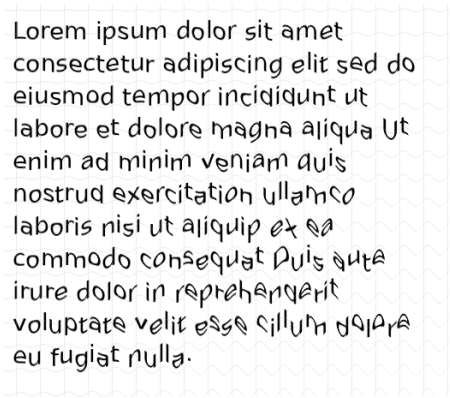
The first panel shows a straight, dense square grid over the original. Each output uses one vertical_wave transform on both the text and this grid; the lines bend or translate with the text. The plain, ungridded output remains available to CLI and script users. Within these panels only px intensity changes. Strong settings may reduce legibility.



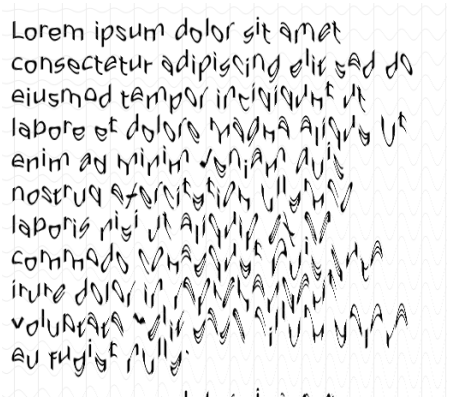
Original + straight square grid



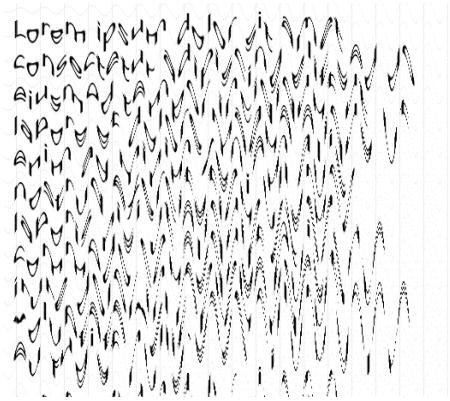
2 px



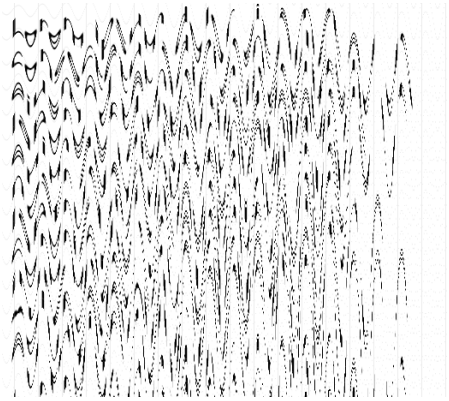
8 px



24 px



60 px



120 px

9.2 Run vertical wave

a. CLI

```
python -m src --input original.png \
  --output vertical_wave.png --page-warp vertical_wave --wavelength 28 \
  --intensity 20
```

b. in script

```
transform = WarpTransform(mode='vertical_wave', wavelength=28, intensity=20)
save('vertical_wave.png', transform(source))
```

To reproduce the compact 4-by-4 study, use the mini-grid.png source from section 4.1. Rows use 14, 28, 42, 56 px wavelengths; columns use 0, 12, 28, 50 px intensity. These commands apply the built-in transform to the text and straight grid together.

c. CLI: four-by-four study

```
for w in 14 28 42 56; do
  for i in 0 12 28 50; do
    python -m src --input mini-grid.png \
      --page-warp vertical_wave --wavelength $w \
      --intensity $i --output study_vertical_wave_${w}_${i}.png
  done
done
```

d. in script: four-by-four study

```
small = image_to_tensor(Image.open('mini-grid.png'))
for w in (14, 28, 42, 56):
    for i in (0, 12, 28, 50):
        transform = WarpTransform(mode='vertical_wave',
            wavelength=w, intensity=i)
        save(f'study_vertical_wave_{w}_{i}.png',
             transform(small))
```

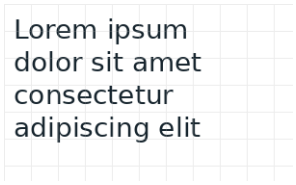
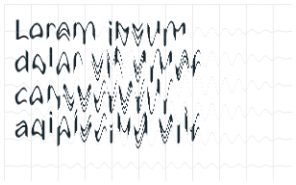
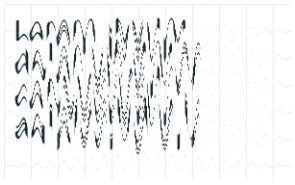
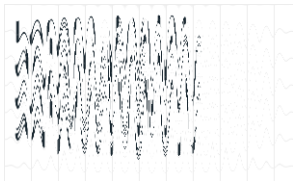
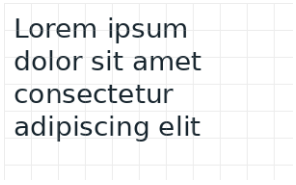
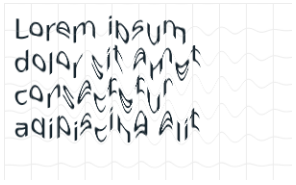
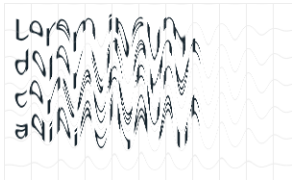
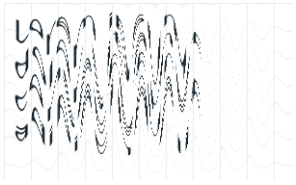
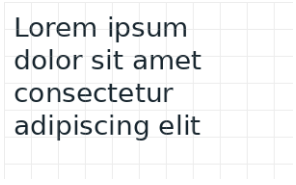
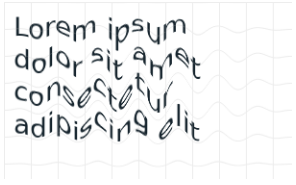
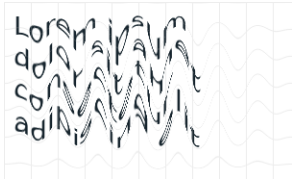
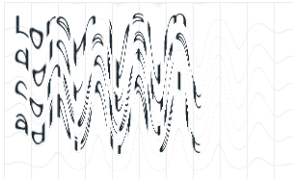
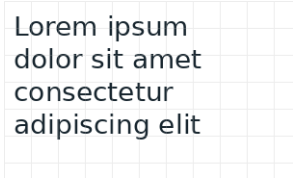
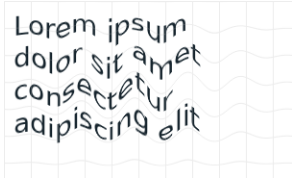
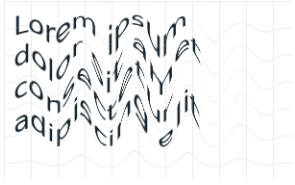
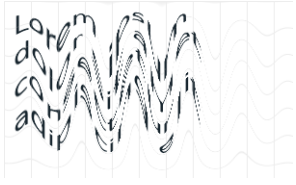
The command writes a PNG. On PowerShell, enter the CLI command on one line. Do not use Unix backslash line continuation there.

9.3 Two-variable legibility study

Rows change X wavelength (px); columns change intensity (px). The 4-by-4 table uses the same four-line source paragraph, unchanged in font, size, and style. Each label lists the row value, then the column value. Compare the letters, spaces, and repeated strokes.

Every cell starts with the same dense square reference grid. The transform warps the grid with the text; zero-intensity cells keep straight lines. These lines do not represent droplet centres.

Each row uses a pixel wavelength calculated from the 28 px source font: 50%, 100%, 150%, and 200% give 14, 28, 42, and 56 px. The built-in wave renders both text and grid with that wavelength; columns vary pixel intensity. The following five-by-five study extends the sweep to 250% and 50 px intensity.

 14 px | 0 px	 14 px | 12 px	 14 px | 28 px	 14 px | 50 px
 28 px | 0 px	 28 px | 12 px	 28 px | 28 px	 28 px | 50 px
 42 px | 0 px	 42 px | 12 px	 42 px | 28 px	 42 px | 50 px
 56 px | 0 px	 56 px | 12 px	 56 px | 28 px	 56 px | 50 px

9.4 Two-parameter wavelength study: Vertical wave

The unchanged input has a 28 px body font. Its 50%, 100%, 150%, 200%, and 250% periods are 14, 28, 42, 56, 70 px. Columns vary wavelength; rows vary intensity: 10, 20, 30, 40, 50 px. Each panel crops the same region of a full-page warp.

The built-in wave uses the stated pixel wavelength and keeps the page-edge envelope fixed.

a. CLI (POSIX shell)

```
for w in 14 28 42 56 70; do
  for i in 10 20 30 40 50; do
    python -m src --input original.png \
      --output vertical_wave_${w}_${i}.png \
      --page-warp vertical_wave --wavelength $w --intensity $i
  done
done
```

b. in script

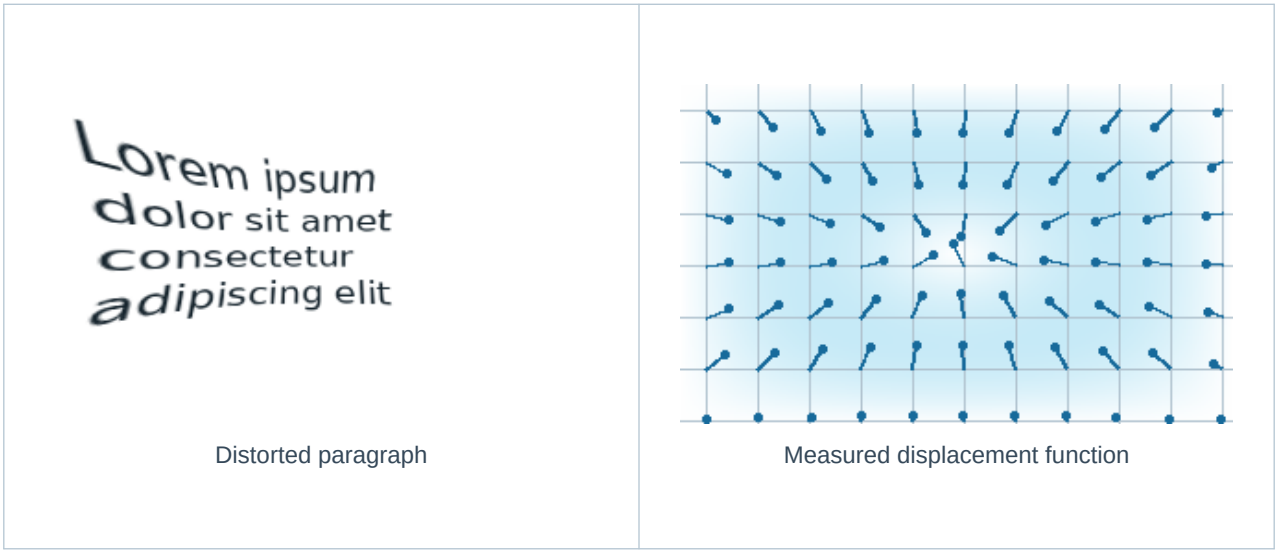
```
for w in (14, 28, 42, 56, 70):
    for i in (10, 20, 30, 40, 50):
        save(f'vertical_wave_{w}_{i}.png',
             WarpTransform(mode='vertical_wave',
                 wavelength=w, intensity=i)(source))
```



All row and column values appear on this page. Strong settings can fold or repeat letters.

10. How gentle pinch works

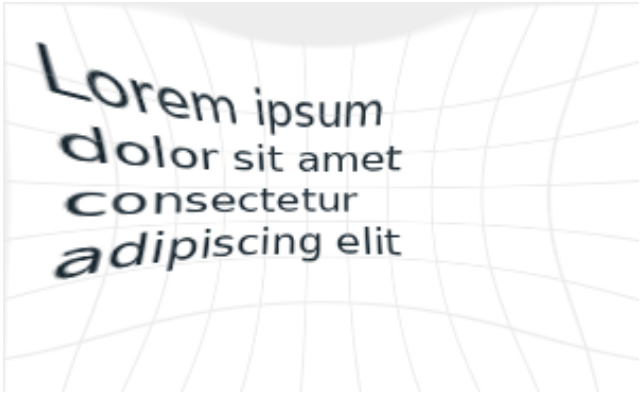
The field points toward the page centre. A smooth envelope holds page edges fixed. The built-in transform exposes pixel intensity, but not a separate width or frequency control.



Distorted paragraph

Measured displacement function

The left panel uses intensity 150 px. The plot shows its actual pixel displacement on a square sampling grid. Arrow lengths are amplified for display. The plot is not an extra transformation.

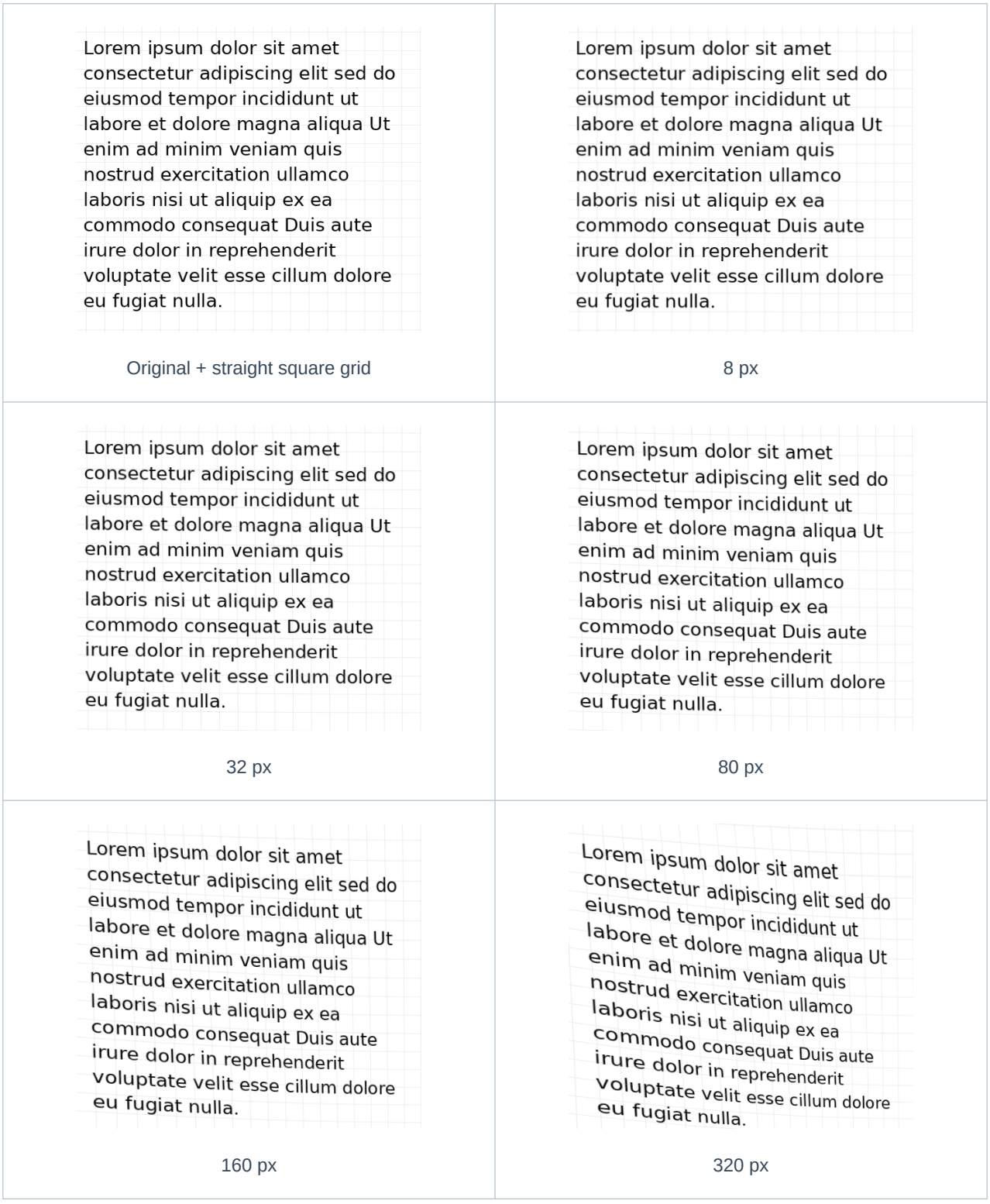


The small overlay above uses a dense square reference grid to show how this effect samples the image. It is not a square array of droplet centres. For the slab, the two mode translations remain constant across the page.

10.1 Gentle pinch: strength examples

A gentle pinch moves text toward the page centre. Its field is small. Use a larger pixel intensity to see the effect.

The first panel shows a straight, dense square grid over the original. Each output uses one gentle_pinch transform on both the text and this grid; the lines bend or translate with the text. The plain, ungridded output remains available to CLI and script users. Within these panels only px intensity changes. Strong settings may reduce legibility.



10.2 Run gentle pinch

a. CLI

```
python -m src --input original.png \
  --output gentle_pinch.png --page-warp gentle_pinch \
  --intensity 160
```

b. in script

```
transform = WarpTransform(mode='gentle_pinch', intensity=160)
save('gentle_pinch.png', transform(source))
```

The compact 4-by-4 pinch study varies a Python-only envelope power; the built-in CLI has intensity but no envelope-power option. Recreate mini-grid.png with section 4.1 and use this custom field for that study.

c. in script: envelope-power study

```
import torch
from functools import partial
def pinch_power_field(y, x, power):
    edge = (torch.sin(torch.pi*y) *
            torch.sin(torch.pi*x)).clamp_min(0)
    edge = edge.pow(power)
    return edge*(.5-y), edge*(.5-x)
small = image_to_tensor(Image.open('mini-grid.png'))
for power in (.6, 1, 1.5, 2):
    for i in (0, 90, 180, 300):
        field = partial(pinch_power_field, power=power)
        save(f'pinch_{power}_{i}.png',
             WarpTransform(mode='custom', intensity=i,
                           warp_function=field)(small))
```

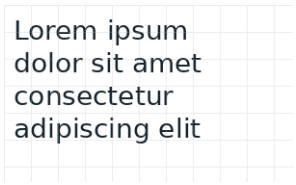
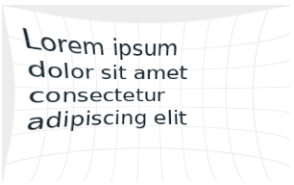
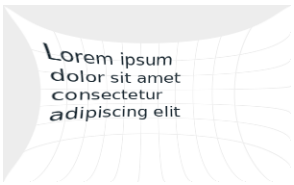
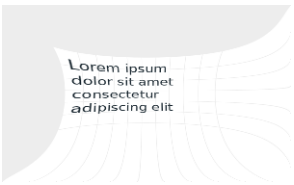
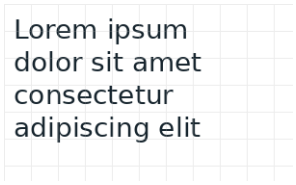
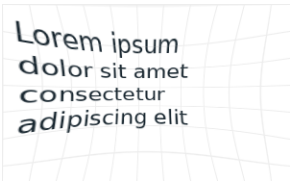
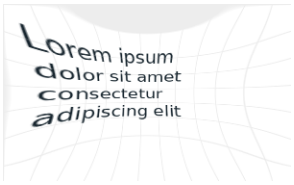
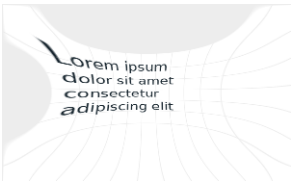
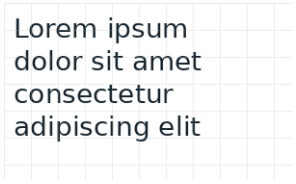
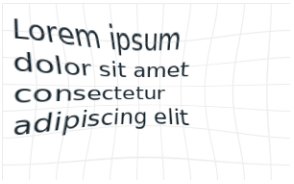
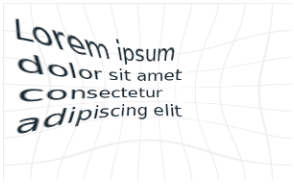
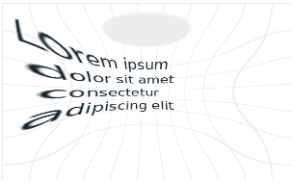
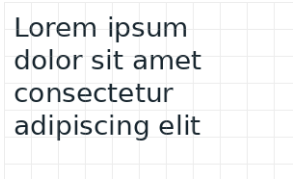
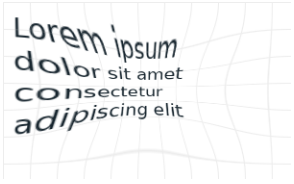
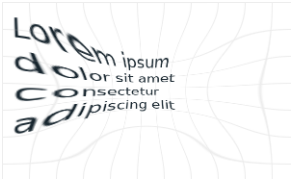
The command writes a PNG. On PowerShell, enter the CLI command on one line. Do not use Unix backslash line continuation there.

10.3 Two-variable legibility study

Rows change pinch envelope power; columns change intensity (px). The 4-by-4 table uses the same four-line source paragraph, unchanged in font, size, and style. Each label lists the row value, then the column value. Compare the letters, spaces, and repeated strokes.

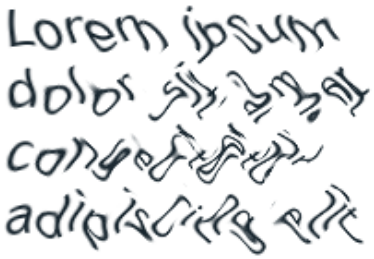
Every cell starts with the same dense square reference grid. The transform warps the grid with the text; zero-intensity cells keep straight lines. These lines do not represent droplet centres.

This study varies the Python-only pinch-envelope power and pixel intensity. The text itself never changes.

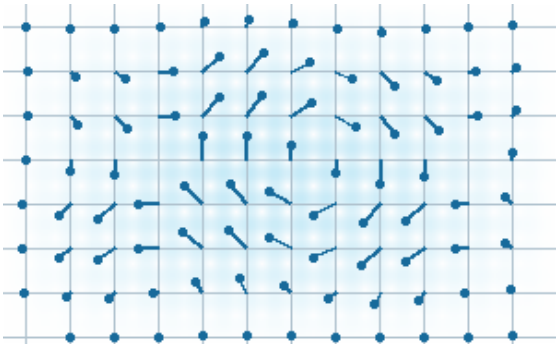
 0.6 | 0	 0.6 | 90	 0.6 | 180	 0.6 | 300
 1 | 0	 1 | 90	 1 | 180	 1 | 300
 1.5 | 0	 1.5 | 90	 1.5 | 180	 1.5 | 300
 2 | 0	 2 | 90	 2 | 180	 2 | 300

11. How custom wave (python only) works

The callback combines vertical (X-periodic) and horizontal (Y-periodic) waves into a (dy, dx) field. Choose a wavelength in pixels and fix one wave at 25% of its page dimension, or use the same wavelength on X and Y. Pixel intensity multiplies both components. Only the Python API accepts the callback.

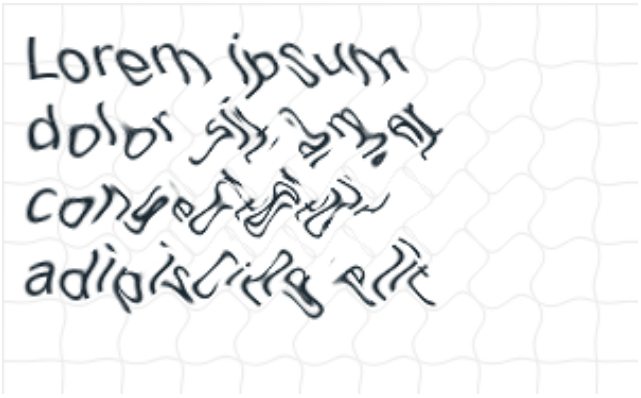


Distorted paragraph



Measured displacement function

The left panel uses X/Y wavelength 28/28 px; intensity 8 px. The plot shows its actual pixel displacement on a square sampling grid. Arrow lengths are amplified for display. The plot is not an extra transformation.

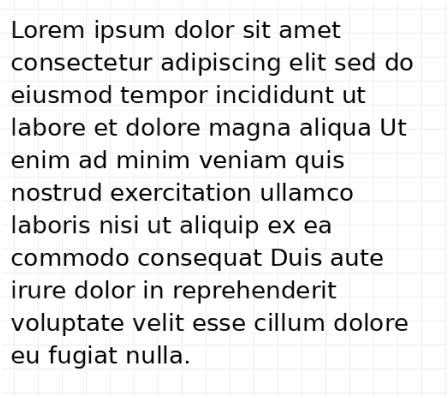


The small overlay above uses a dense square reference grid to show how this effect samples the image. It is not a square array of droplet centres. For the slab, the two mode translations remain constant across the page.

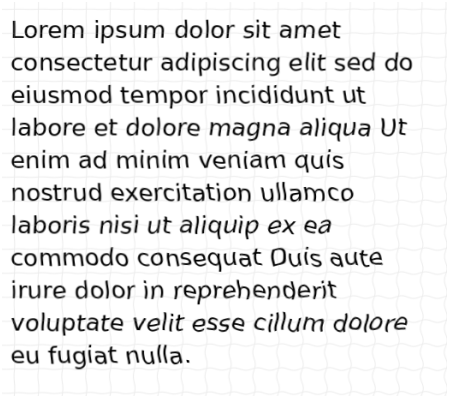
11.1 Custom wave (Python only): strength examples

A custom wave returns Y and X displacement tensors. The gallery holds a symmetric 28 px wavelength in both axes and varies only intensity. Separate studies compare fixed vertical, fixed horizontal, and symmetric wavelengths. Custom callbacks are Python-only.

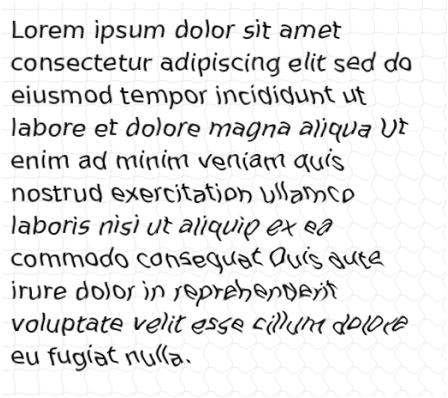
The first panel shows a straight, dense square grid over the original. Each output uses one custom transform on both the text and this grid; the lines bend or translate with the text. The plain, ungridded output remains available to CLI and script users. Within these panels only px intensity changes. Strong settings may reduce legibility.



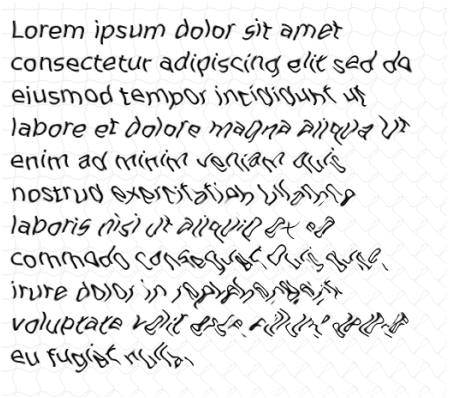
Original + straight square grid



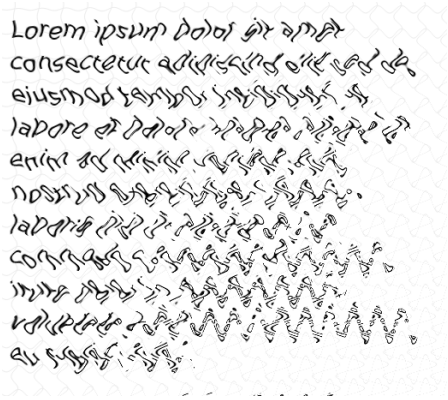
2 px



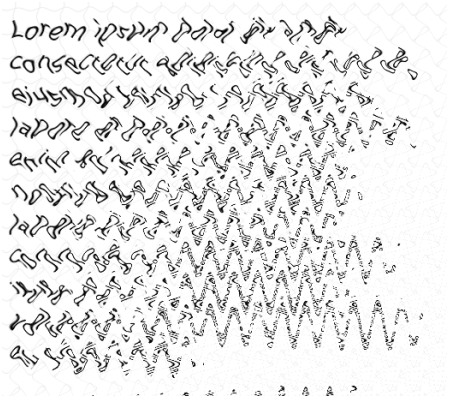
5 px



10 px



20 px



30 px

11.2 Run custom wave (python only)

The custom mode has no CLI option. Set wavelength=28 pixels and intensity=20 pixels for each of the three variants. A fixed vertical wave has a 25% of image width period; a fixed horizontal wave has a 25% of image height period. Symmetric uses 28 px on both axes.

a. in script

```
from functools import partial
from src import custom_wave
for variant in ('vertical', 'horizontal', 'symmetric'):
    field = partial(custom_wave, wavelength=28,
                    fixed_axis=variant)
    transform = WarpTransform(mode='custom', intensity=20,
                              warp_function=field)
    save('custom_' + variant + '.png', transform(source))
```

The compact four-by-four studies below use an independent X/Y-period callback. Recreate its fixed four-line input with section 4.1. The callback receives normalized (y, x) coordinates and returns unitless (dy, dx) components; WarpTransform applies pixel intensity. No CLI equivalent exists for this two-axis callback.

b. in script: independent X/Y study

```
import torch
from functools import partial
def xy_wave(y, x, wx, wy):
    edge = torch.sin(torch.pi*y) * torch.sin(torch.pi*x)
    dy = edge * torch.sin(2*torch.pi*x*299/wx)
    dx = edge * torch.sin(2*torch.pi*y*183/wy)
    return dy, dx
small = image_to_tensor(Image.open('mini-grid.png'))
for strength in (5, 10, 20):
    for wx in (14, 28, 42, 56):
        for wy in (14, 28, 42, 56):
            field = partial(xy_wave, wx=wx, wy=wy)
            transform = WarpTransform(mode='custom',
                intensity=strength, warp_function=field)
            save(f'xy_{strength}_{wx}_{wy}.png',
                 transform(small))
```

The five-by-five wavelength-by-intensity studies later in this chapter use the supplied custom_wave callback.

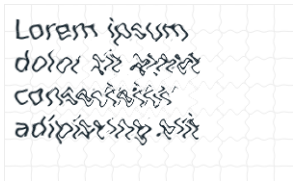
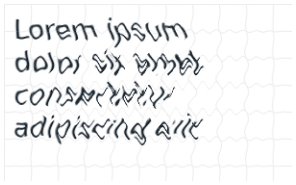
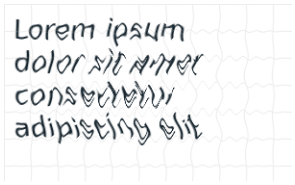
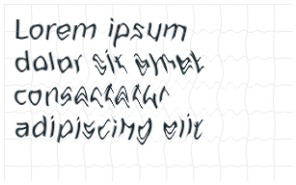
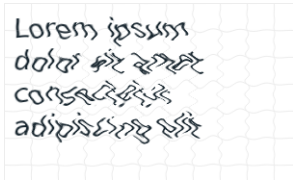
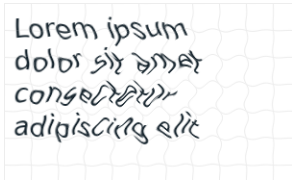
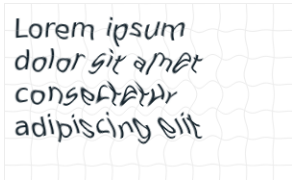
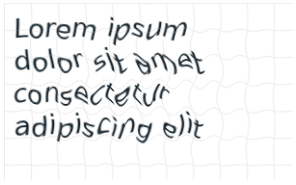
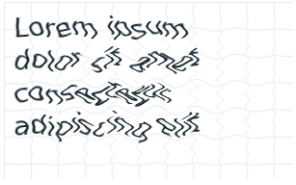
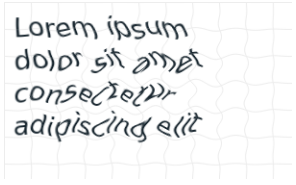
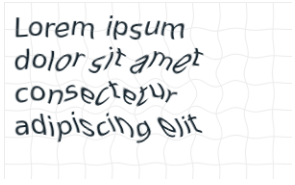
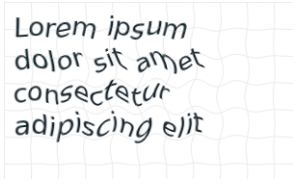
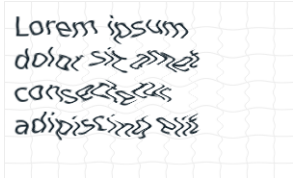
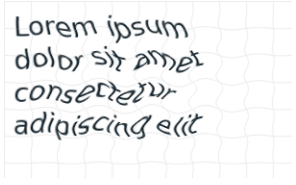
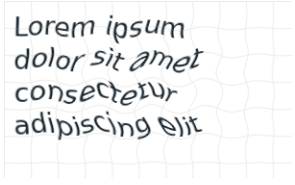
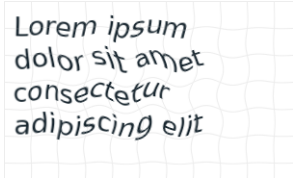
The script writes a PNG. Its callback is Python-only; no equivalent CLI option exists.

11.3 Two-variable legibility study: 5 px

Rows change X wavelength (px); columns change Y wavelength (px). The 4-by-4 table uses the same four-line source paragraph, unchanged in font, size, and style. Each label lists the row value, then the column value. Compare the letters, spaces, and repeated strokes.

The text and straight source grid stay fixed. Each output uses the same Python-only callback formula on both text and grid; only its X and Y pixel wavelengths change. Every cell is warped; no zero-intensity control appears on this page.

Keep pixel intensity at 5 throughout this page. X wavelength runs 14, 28, 42, and 56 px down the rows; Y wavelength runs 14, 28, 42, and 56 px across the columns. These values are 50%, 100%, 150%, and 200% of the 28 px source font. This independently parameterized callback is Python-only.

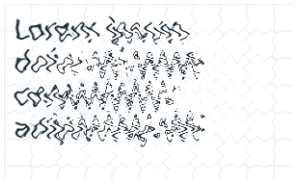
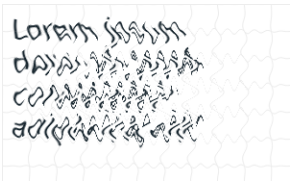
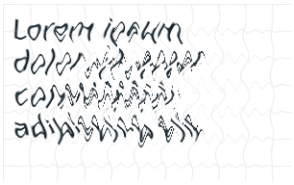
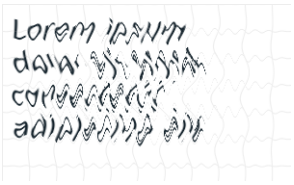
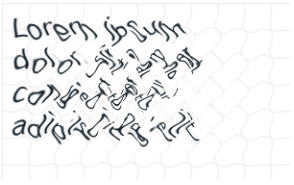
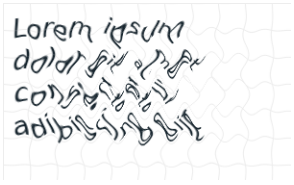
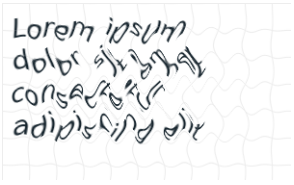
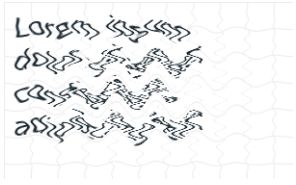
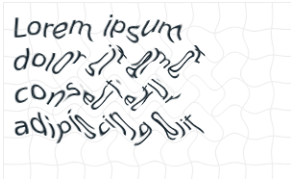
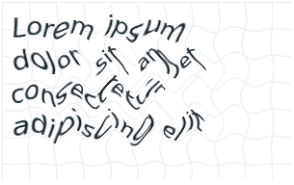
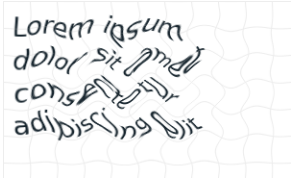
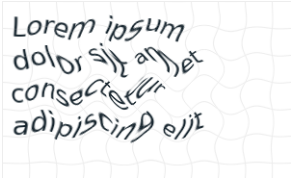
 14 px | 14 px	 14 px | 28 px	 14 px | 42 px	 14 px | 56 px
 28 px | 14 px	 28 px | 28 px	 28 px | 42 px	 28 px | 56 px
 42 px | 14 px	 42 px | 28 px	 42 px | 42 px	 42 px | 56 px
 56 px | 14 px	 56 px | 28 px	 56 px | 42 px	 56 px | 56 px

11.3 Two-variable legibility study: 10 px

Rows change X wavelength (px); columns change Y wavelength (px). The 4-by-4 table uses the same four-line source paragraph, unchanged in font, size, and style. Each label lists the row value, then the column value. Compare the letters, spaces, and repeated strokes.

The text and straight source grid stay fixed. Each output uses the same Python-only callback formula on both text and grid; only its X and Y pixel wavelengths change. Every cell is warped; no zero-intensity control appears on this page.

Keep pixel intensity at 10 throughout this page. X wavelength runs 14, 28, 42, and 56 px down the rows; Y wavelength runs 14, 28, 42, and 56 px across the columns. These values are 50%, 100%, 150%, and 200% of the 28 px source font. This independently parameterized callback is Python-only.

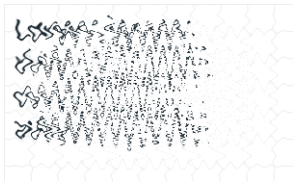
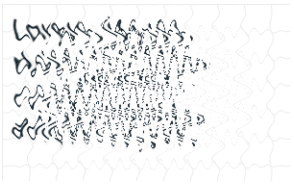
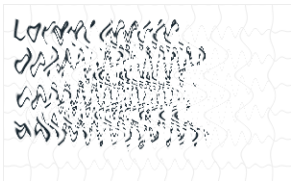
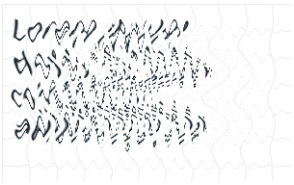
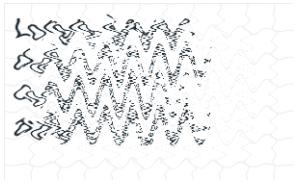
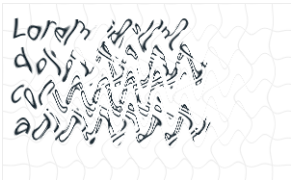
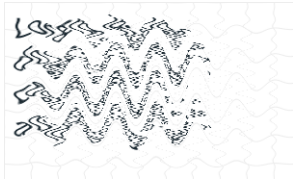
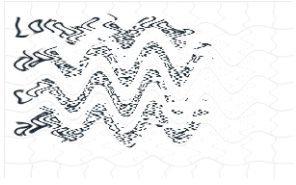
 14 px | 14 px	 14 px | 28 px	 14 px | 42 px	 14 px | 56 px
 28 px | 14 px	 28 px | 28 px	 28 px | 42 px	 28 px | 56 px
 42 px | 14 px	 42 px | 28 px	 42 px | 42 px	 42 px | 56 px
 56 px | 14 px	 56 px | 28 px	 56 px | 42 px	 56 px | 56 px

11.3 Two-variable legibility study: 20 px

Rows change X wavelength (px); columns change Y wavelength (px). The 4-by-4 table uses the same four-line source paragraph, unchanged in font, size, and style. Each label lists the row value, then the column value. Compare the letters, spaces, and repeated strokes.

The text and straight source grid stay fixed. Each output uses the same Python-only callback formula on both text and grid; only its X and Y pixel wavelengths change. Every cell is warped; no zero-intensity control appears on this page.

Keep pixel intensity at 20 throughout this page. X wavelength runs 14, 28, 42, and 56 px down the rows; Y wavelength runs 14, 28, 42, and 56 px across the columns. These values are 50%, 100%, 150%, and 200% of the 28 px source font. This independently parameterized callback is Python-only.

 14 px | 14 px	 14 px | 28 px	 14 px | 42 px	 14 px | 56 px
 28 px | 14 px	 28 px | 28 px	 28 px | 42 px	 28 px | 56 px
 42 px | 14 px	 42 px | 28 px	 42 px | 42 px	 42 px | 56 px
 56 px | 14 px	 56 px | 28 px	 56 px | 42 px	 56 px | 56 px

11.4 Two-parameter wavelength study: Fixed vertical custom wave

The unchanged input has a 28 px body font. Its 50%, 100%, 150%, 200%, and 250% periods are 14, 28, 42, 56, 70 px. Columns vary wavelength; rows vary intensity: 10, 20, 30, 40, 50 px. Each panel crops the same region of a full-page warp.

The vertical component has a fixed period of 25% of image width (300 px). The horizontal component uses the column wavelength along Y.

Custom callbacks have no CLI mode. Run this Python-only script after the setup in section 4.

a. in script

```
from functools import partial
from src import custom_wave
for w in (14, 28, 42, 56, 70):
    for i in (10, 20, 30, 40, 50):
        field = partial(custom_wave, wavelength=w,
                        fixed_axis='vertical')
        save(f'custom_vertical_{w}_{i}.png',
             WarpTransform(mode='custom', intensity=i,
                           warp_function=field)(source))
```



All row and column values appear on this page. Strong settings can fold or repeat letters.

11.5 Two-parameter wavelength study: Fixed horizontal custom wave

The unchanged input has a 28 px body font. Its 50%, 100%, 150%, 200%, and 250% periods are 14, 28, 42, 56, 70 px. Columns vary wavelength; rows vary intensity: 10, 20, 30, 40, 50 px. Each panel crops the same region of a full-page warp.

The horizontal component has a fixed period of 25% of image height (400 px). The vertical component uses the column wavelength along X.

Custom callbacks have no CLI mode. Run this Python-only script after the setup in section 4.

a. in script

```
from functools import partial
from src import custom_wave
for w in (14, 28, 42, 56, 70):
    for i in (10, 20, 30, 40, 50):
        field = partial(custom_wave, wavelength=w,
                        fixed_axis='horizontal')
        save(f'custom_horizontal_{w}_{i}.png',
             WarpTransform(mode='custom', intensity=i,
                           warp_function=field)(source))
```



All row and column values appear on this page. Strong settings can fold or repeat letters.

11.6 Two-parameter wavelength study: Symmetric custom wave

The unchanged input has a 28 px body font. Its 50%, 100%, 150%, 200%, and 250% periods are 14, 28, 42, 56, 70 px. Columns vary wavelength; rows vary intensity: 10, 20, 30, 40, 50 px. Each panel crops the same region of a full-page warp.

X and Y have the same pixel wavelength in every column.

Custom callbacks have no CLI mode. Run this Python-only script after the setup in section 4.

a. in script

```
from functools import partial
from src import custom_wave
for w in (14, 28, 42, 56, 70):
    for i in (10, 20, 30, 40, 50):
        field = partial(custom_wave, wavelength=w,
                        fixed_axis='symmetric')
        save(f'custom_symmetric_{w}_{i}.png',
             WarpTransform(mode='custom', intensity=i,
                           warp_function=field)(source))
```

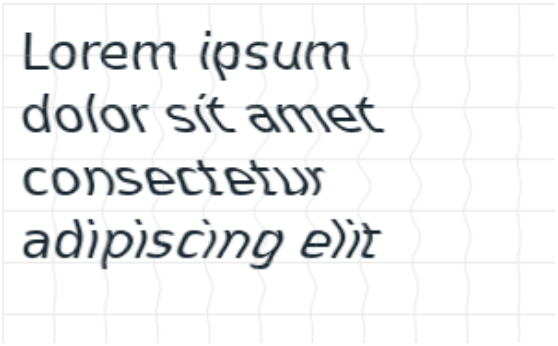


All row and column values appear on this page. Strong settings can fold or repeat letters.

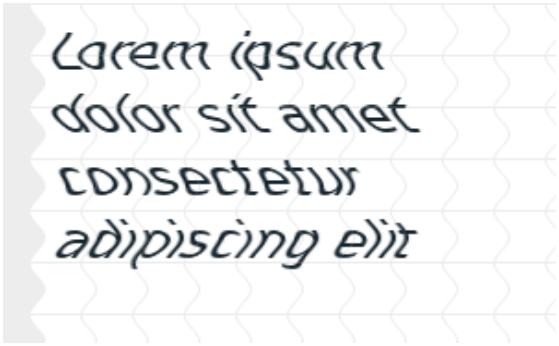
12. Edge taper and pixel shifts

Compare the same four-line text and straight reference grid in all four panels. Use horizontal_wave, wavelength 28 px, and intensity 4 px. The first panel uses edge_taper=True and zero shifts: wave displacement is exactly 0 px at each page edge. X and Y are right and down. Negative values move left and up.

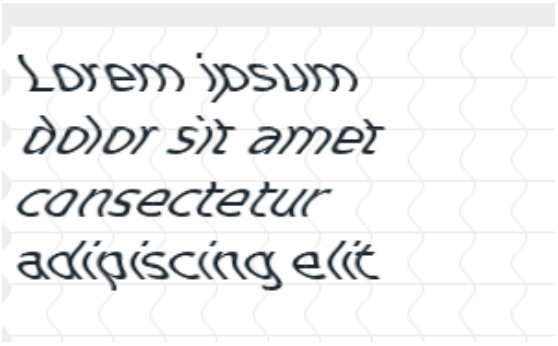
A nonzero shift_X or shift_Y overrides edge_taper=True: the wave itself becomes untapered before the pixel translation is added. These shifts are absolute pixels, not multiples of intensity. Both the wave and the translation are scaled by a displacement mask if one is supplied. Droplets and physical refraction do not accept these page-warp controls.



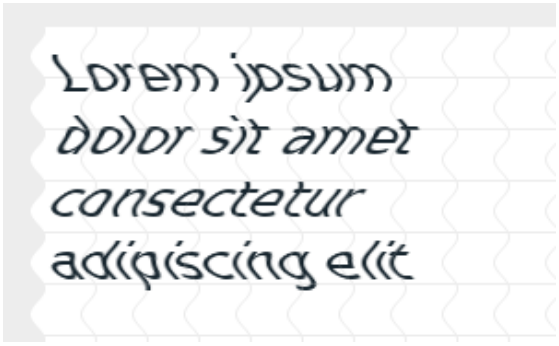
edge_taper=True
shift_X=0 px; shift_Y=0 px



X shifted transform
shift_X=18 px; shift_Y=0 px



Y shifted transform
shift_X=0 px; shift_Y=12 px



XY shifted transform
shift_X=18 px; shift_Y=12 px

Create mini-grid.png with section 4.1. All four images above use that same input; only the two shift values change. The code explicitly requests tapering so the shift override is visible.

a. CLI: XY shift

```
python -m src --input mini-grid.png --output shift-xy.png \
  --page-warp horizontal_wave --wavelength 28 \
  --intensity 4 --edge-taper --shift-X 18 --shift-Y 12
```

b. in script: all four panels

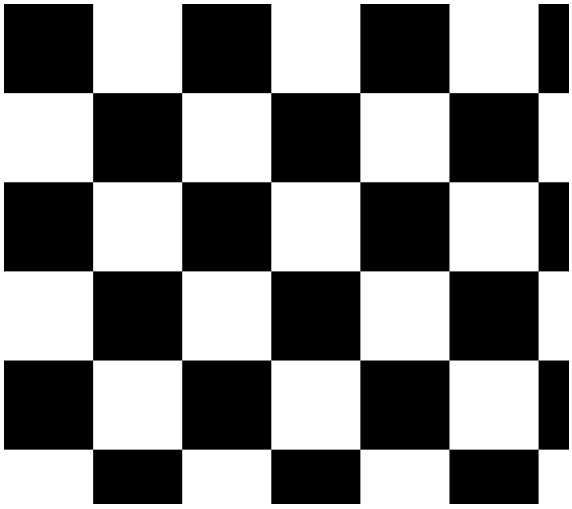
```
small = image_to_tensor(Image.open('mini-grid.png'))
for name, dx, dy in (('taper', 0, 0), ('x', 18, 0),
                     ('y', 0, 12), ('xy', 18, 12)):
    transform = WarpTransform(mode='horizontal_wave',
        wavelength=28, intensity=4, edge_taper=True,
        shift_X=dx, shift_Y=dy)
    save('shift-' + name + '.png', transform(small))
```

13. Image Transform

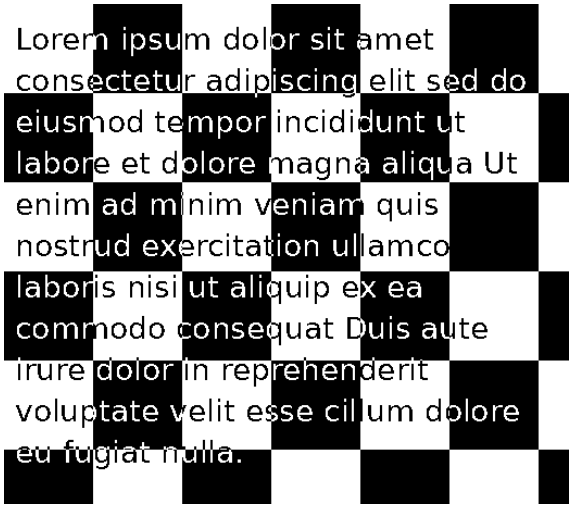
An image transform changes pixel values. It does not move pixels. Use ImageTransform in Python to apply logical AND, OR, or XOR with a same-size binary mask. The main CLI --mask option controls a warp's displacement instead; it cannot select a logical image operation.

13.1 XOR

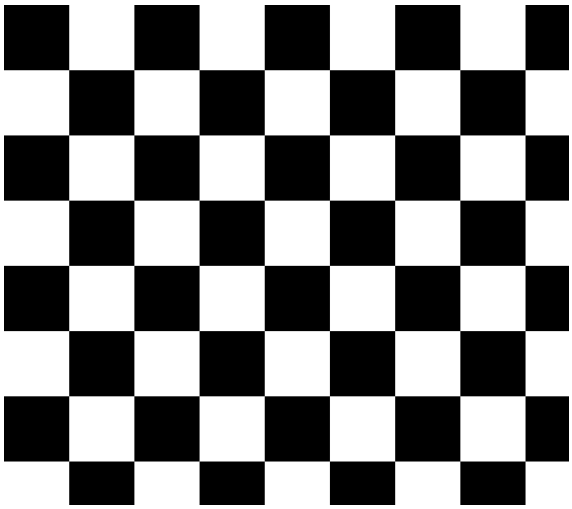
A white mask square (1) inverts black and white. A black square (0) keeps the image. The source is the same 520-by-460 crop of the generated page in all six examples. Its body font is 28 px. Each result comes from the saved checkerboard mask and the same source, not a separate warp. Gray image values are thresholded at 0.5; gray masks are invalid.



Example 02: 82 px squares | mask



Example 02: 82 px squares | XOR output

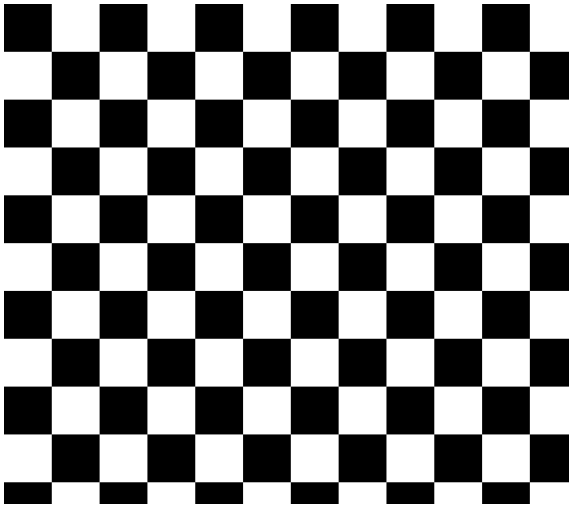


Example 03: 60 px squares | mask

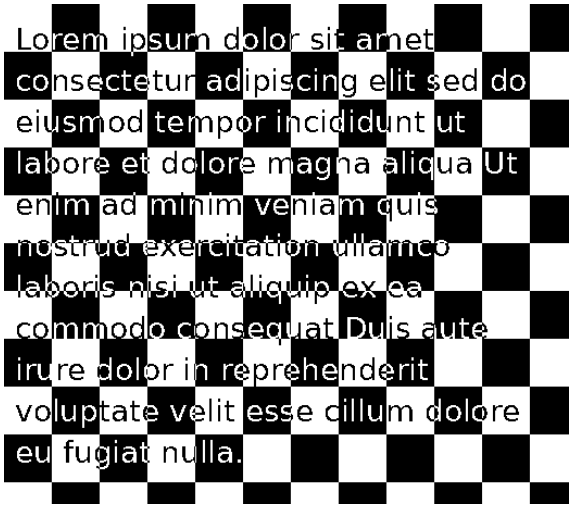


Example 03: 60 px squares | XOR output

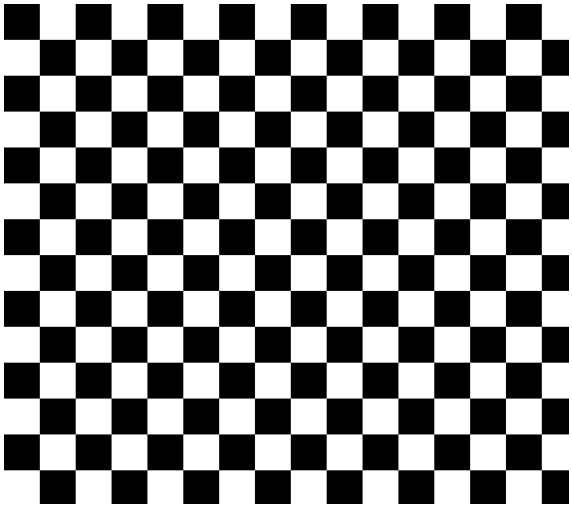
13.1 XOR (continued). The left panel is the binary mask; the right panel is the saved XOR result. Black mask squares keep the source. White mask squares invert it.



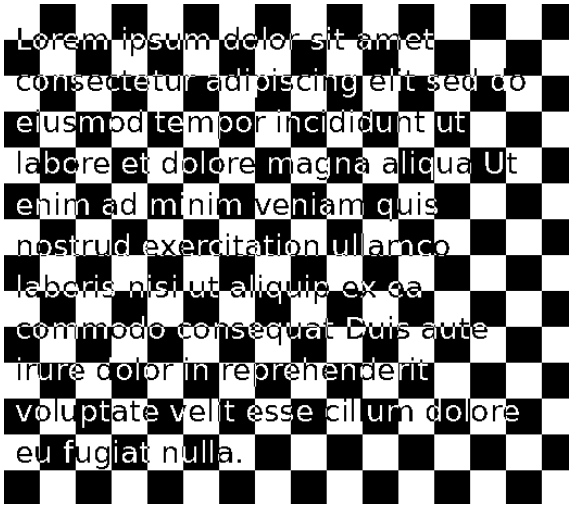
Example 04: 44 px squares | mask



Example 04: 44 px squares | XOR output

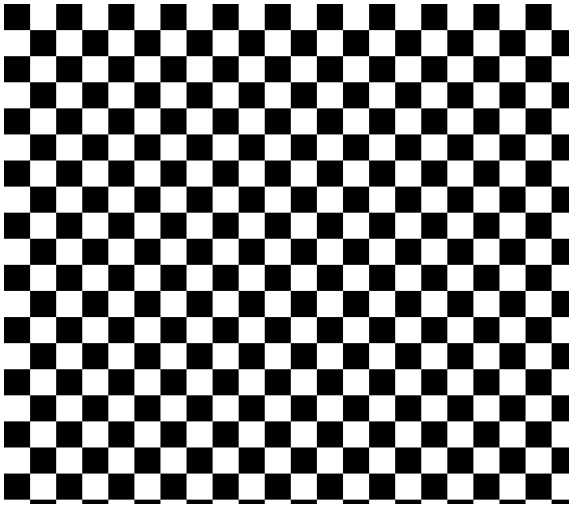


Example 05: 33 px squares | mask

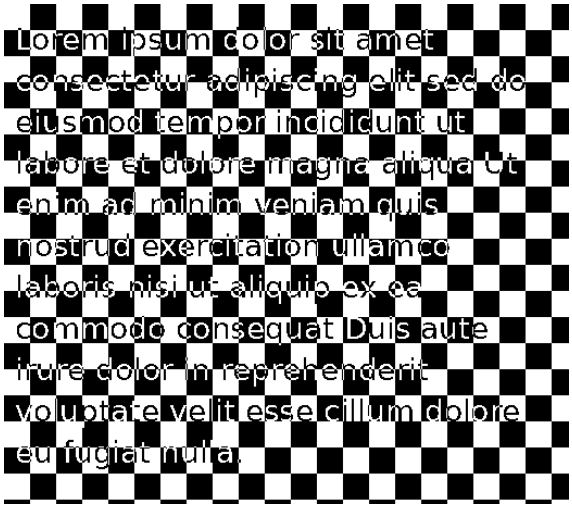


Example 05: 33 px squares | XOR output

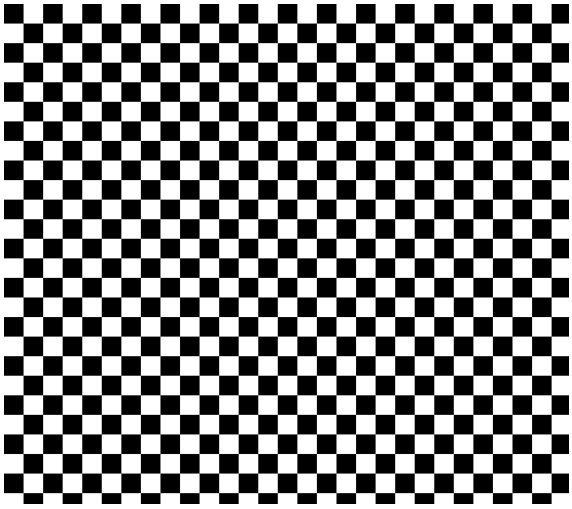
13.1 XOR (continued). The left panel is the binary mask; the right panel is the saved XOR result. Black mask squares keep the source. White mask squares invert it.



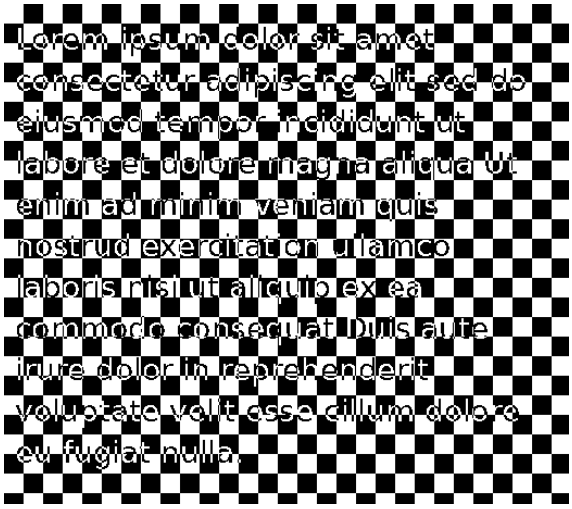
Example 06: 24 px squares | mask



Example 06: 24 px squares | XOR output



Example 07: 18 px squares | mask



Example 07: 18 px squares | XOR output

Create the source page with section 3. Crop it to (70, 104, 590, 564). Each mask uses (x // square + y // square) % 2. Set its square size in pixels. Pass the original cropped image and that mask to ImageTransform.logical_xor; do not pass the earlier XOR result to the next example.

a. in script: generate the six results

```
import numpy as np
from src import ImageTransform, image_to_tensor
small = Image.open('original.png').crop((70,104,590,564))
source = image_to_tensor(small)
h, w = source.shape[1:]
y, x = np.indices((h, w))
for number, size in ((2,82),(3,60),(4,44),
                     (5,33),(6,24),(7,18)):
    checks = ((x//size + y//size) % 2).astype('uint8')
    mask = Image.fromarray(checks * 255, 'L')
    mask.save(f'checks-{number:02d}.png')
    result = ImageTransform.logical_xor(
        source, f'checks-{number:02d}.png')
    save(f'xor-{number:02d}.png', result)
```